DEVELOPER NOTES

# TypeScript

## PREMIUM NOTES

Fundamentals • Advanced Concepts • Backend • React

A structured, practical reference for modern TypeScript development

```
const developer = {
  language: "TypeScript"
  level:      "Practical"
```

# TYPESCRIPT PREMIUM NOTES

COMPLETE TOPIC INDEX


| NO. | TOPIC | SOURCE PAGE |
|---|---|---|
| 1 | TypeScript Introduction | 1 |
| 2 | Project Setup & Installation | 2 |
| 3 | tsconfig.json & Important Features | 3 |
| 4 | Direct File Execution (tsx / ts-node) | 4 |
| 5 | Running First TypeScript Program | 5 |
| 6 | Core Types: Number, String, Boolean | 6 |
| 7 | Arrays, Objects & Tuples | 7 |
| 8 | Enum, Any & Unknown | 8 |
| 9 | Void, Null, Never & Undefined | 9 |
| 10 | BigInt | 10 |
| 11 | Type Inference & Type Assertions | 11 |
| 12 | Union Types & Type Narrowing | 12 |
| 13 | Interfaces | 13 |
| 14 | Interface Properties & Function Interfaces | 14 |
| 15 | Extending Interfaces | 15 |
| 16 | Type Aliases | 16 |
| 17 | Intersection & Literal Types | 17 |
| 18 | Advanced Type Aliases | 18 |
| 19 | Classes in TypeScript | 19 |
| 20 | Access Modifiers | 20 |
| 21 | Inheritance | 21 |
| 22 | Abstract Classes | 22 |
| 23 | Generics | 23 |
| 24 | Generics with Type Aliases | 24 |
| 25 | Type Guards | 25 |


*Page numbers refer to the supplied TypeScript notes.*

# TYPESCRIPT PREMIUM NOTES

COMPLETE TOPIC INDEX


| NO. | TOPIC | SOURCE PAGE |
|---|---|---|
| 26 | keyof Operator | 26 |
| 27 | Index Signatures | 27 |
| 28 | Utility Types | 28 |
| 29 | Decorators | 29 |
| 30 | Non-Null Assertion Operator | 30 |
| 31 | Backend Development with TypeScript | 31 |
| 32 | Backend Development: Script & Package Setup | 32 |
| 33 | Backend Development: npm Scripts & TypeScript Build | 33 |
| 34 | Backend Development: Project Structure | 34 |
| 35 | Backend Development: Running the Backend | 35 |
| 36 | Backend Development: nodemon / Development Workflow | 36 |
| 37 | Backend Development: Server & API Basics | 37 |
| 38 | Type Definition Package: @types/express | 38 |
| 39 | .env Files & Backend Configuration | 39 |
| 40 | MongoDB Connectivity | 40 |
| 41 | MongoDB Model (Mongoose + TypeScript) | 41 |
| 42 | Controller / Business Logic | 42 |
| 43 | TypeScript Import / Export | 43 |
| 44 | React Project Setup with TypeScript | 44 |
| 45 | React Props with Proper Types | 45 |
| 46 | Alternative Props Type Examples | 46 |
| 47 | State Type & Event Object Type Management | 47 |
| 48 | Function Component Props with Children | 48 |
| 49 | Props Sending Function Type / Safety | 49 |


*Page numbers refer to the supplied TypeScript notes.*

# TYPESCRIPT FUNDAMENTALS NOTES

## Page 1: Typescript Introduction

Beginner to Intermediate

### ★ 1. What is TypeScript?

- TypeScript is a statically typed superset of JavaScript developed by Microsoft.
- It adds optional static typing, classes, interfaces, and other modern features to JavaScript.
- It helps developers write more maintainable, scalable and error-free code.
- TypeScript code is compiled to plain JavaScript, so it runs in any browser or Node.js environment.

**Key Points**

- **Developed by:** Microsoft
- **Superset of:** JavaScript
- **Adds:** Static typing, interfaces, classes, etc.
- **Compiles to:** JavaScript (ES5/ES6)
- **Runs in:** Browsers & Node.js

### ★ 2. Why Use TypeScript?

- Catch errors early (during development).
- Better code completion in IDEs.
- Improved code readability and maintainability.
- Supports modern JavaScript features.
- Great for large scale applications.

**Popular Features**

- Static Typing
- Interfaces
- Classes
- Enums
- Generics
- Modules

### ★ 3. TypeScript vs JavaScript

| TypeScript | JavaScript |
|---|---|
| Static typing | Dynamic typing |
| Compile time errors | Runtime errors |
| Better IDE support | Limited IDE support |
| More structure | Less structure |
| Larger codebase | Smaller codebase |

**Example**

```
// TypeScript
let message: string = "Hello, TypeScript!";
console.log(message);

// Compiled to JavaScript
var message = "Hello, TypeScript!";
console.log(message);
```

> "TypeScript doesn't replace JavaScript, it makes JavaScript better!

Better Code Brighter Future

# TYPESCRIPT PROJECT SETUP

## Using Node / NPM and Installation of TypeScript

### Page 2: Project Setup & Installation

## ★ 1. Prerequisites

- Node.js must be installed on your system.
- NPM (Node Package Manager) comes with Node.js.
- You can check if they are installed using these commands:

```
node -v     # Check Node.js version
npm -v      # Check NPM version
```

**Quick Info**

- Node.js = JavaScript runtime
- NPM = package manager
- Both are required to work with TypeScript.

## ★ 2. Create a New Project Folder

- Make a new folder for your TypeScript project.
- Open terminal in that folder and run:

```
mkdir typescript-app
cd typescript-app
```

← This will create a new folder and move you inside it.

**Project Structure (so far)**

```
typescript-app/
├── node_modules/
├── package.json
└── (we will add more files later)
```

## ★ 3. Initialize a Node Project

- Run the following command to create a package.json file:

```
npm init -y
```

← This will create a default package.json with yes to all questions.

**Important Points**

- Always use Node.js (LTS version).
- Use npm (comes with Node.js).
- --save-dev is used because TypeScript is only needed for development, not production.
- You can also use yarn or pnpm instead of npm (optional).

## ★ 4. Install TypeScript

- Now install TypeScript as a dev dependency:

```
npm install typescript --save-dev
```

- This adds TypeScript to your project's node_modules and package.json.

Example: package.json (partial)

```
{
  "name": "typescript-app",
  "version": "1.0.0",
  "devDependencies": {
    "typescript": "^5.0.0"
  }
}
```

← TypeScript will be listed under devDependencies after installation.

**Common Mistakes**

- Forgetting to install Node.js.
- Running commands outside the project folder.
- Not using --save-dev flag.

> "Set up your environment, build your foundation, then write great code!"

# TYPESCRIPT

## tsconfig.json & Important Features

### Configure Your Project with tsconfig.json

Page 3: tsconfig.json & Important Features

## ★ 1. What is tsconfig.json?

- tsconfig.json is a configuration file for TypeScript.
- It tells the TypeScript compiler how to build your project.
- It helps you set compiler options, include/exclude files, etc.
- If you don't create it, TypeScript uses default settings.

**Quick Note:**
You can create it manually or use:
`npx tsc --init`
This will generate a default tsconfig.json file.

## ★ 2. Example of tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "outDir": "../dist",
    "noEmit": false
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

This file configures the compiler options and file inclusions/exclusions for your project.

## ★ 3. Important Features (Options)

### Key Options and Their Use

| | | |
|---|---|---|
| target | : | Set the JavaScript version (e.g. ES5, ES6). |
| module | : | Module system (e.g. commonjs, ESNext). |
| include | : | Files / folders to include in compilation. |
| exclude | : | Files / folders to ignore. |
| outDir | : | Output directory for compiled files. |
| rootDir | : | Root directory of your source files. |

### Detailed Breakdown

**1. include**
- Specifies which files/folders should be included in the compilation.
- Default: `["**/*"]` (all files)
- Example: `"include": ["src"]`

**2. exclude**
- Specifies which files/folders should be excluded from compilation.
- Example: `"exclude": ["node_modules", "dist"]`

**3. noEmit**
- Tells TypeScript not to emit (generate) JavaScript files.
- Useful when you only want type-checking.
- Example: `"noEmit": true`

**4. strict**
- Enables all strict type-checking options.
- Helps catch errors early.
- Example: `"strict": true`

**5. esModuleInterop**
- Enables compatibility with CommonJS and ES Modules.
- Allows default imports from CommonJS.
- Example: `"esModuleInterop": true`

### Other Useful Options

- allowJs : Allow JavaScript files in project.
- skipLibCheck : Skip type checking for .d.ts files.
- forceConsistentCasingInFileNames : Ensure consistent file name casing.
- resolveJsonModule : Allow importing .json files.

**Pro Tip:**
Start with `"strict": true`, and then relax options if needed. It will help you write safer and cleaner code.

> A well-configured tsconfig.json makes your TypeScript project more powerful and error-free!

# TYPESCRIPT
# DIRECT FILE EXECUTION
## Using tsx / ts-node

Run .ts files directly!

Page 4: Direct File Execution Using tsx / ts-node

Beginner to Intermediate

## ★ 1. What is Direct File Execution?

- Normally, Node.js can only run JavaScript files (.js).
- TypeScript files (.ts) need to be compiled first.
- Tools like ts-node and tsx allow you to run .ts files directly, without a separate build step.
- They handle the compilation in memory and execute the file immediately.

### Quick Note

- ts-node = traditional, slower, more flexible.
- tsx = faster, uses esbuild, recommended for most cases.
- Both work great for development and small scripts.

## ★ 2 Using ts-node

### Step 1: Install ts-node (globally or locally)

```
# Global install (optional)
npm install -g ts-node

# Or lacal (recommended)
npm install --save-dev ts-node
```

This installs ts-node so you can run .ts files directly.

### Step 2: Run your TypeScript file

```
npx ts-node src/index.ts
```

This will compile and execute the file in one step.

### Example: ts-node

```
# src/index.ts
const message: string = "Hello from TS!";
console.log(message);
```

```
# Run
npx ts-node src/index.ts
Hello from TS!
```

## ★ 3. Using tsx (Faster Option)

### Step 1: Install tsx

```
npm install --save-dev tsx
```

tsx uses esbuild (super fast) and works great with ESM & CJS.

### Step 2: Run your TypeScript file

```
npx tsx src/index.ts
```

No extra config needed. Just run and go!

### Comparison

| Feature | ts-node | tsx |
|---|---|---|
| Speed | Slower | Faster |
| Compiler | TypeScript | esbuild |
| ESM Support | Good | Excellent |
| Best for | Development Scripts | Modern Projects |

### Example Project Structure

```
my-ts-app/
├── src/
│   └── index.ts
├── package.json
└── tsconfig.json
```

Keep your project organized for a better development experience.

### Important

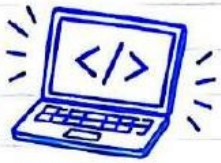
- Make sure to install TypeScript as a dev dependency.
- If using ts-node, you can also run it with:

```
node -r ts-node/register src/index.ts
```

- tsx can also be used with npm scripts.

> "Run your TypeScript files directly, save time, and focus on building, not compiling!"

# TYPESCRIPT
# RUNNING FIRST TS PROGRAM (Hello World)



Page 5: Running Your First TypeScript Program

Beginner to Intermediate

## ★ 1. What We Are Doing?

- Write a simple TypeScript program (Hello World).
- Compile it to JavaScript using tsc.
- Run the compiled file with Node.js.
- See the output in terminal.

### Quick Steps (Summary)

1. Create index.ts in src/
2. Write your TypeScript code
3. Run npx tsc (to compile)
4. Run the compiled file with Node.js
5. See the output

## ★ 2. Step 1: Create a File

- Create a new file in your project folder: src/index.ts

```
my-ts-app/
├─ src/
│  └─ index.ts
├─ package.json
└─ tsconfig.json
```

Your TypeScript code will go here.

## ★ 3. Step 2: Write Hello World in TypeScript

```
// src/index.ts
const message: string = "Hello, TypeScript!";
console.log(message);
```

We also added the type 'string' to show TypeScript's strong typing.

## ★ 4. Step 3: Compile the File

- Run the following command to compile:

```
npx tsc
```

This will create a JavaScript file in the dist folder (default).

```
my-ts-app/
├─ src/
│  └─ index.ts
├─ dist/
│  └─ index.js
├─ package.json
└─ tsconfig.json
```

Compiled JS file will be here.

### 5. Step 4: Run the Compiled File

Use Node.js to run the JavaScript file:

```
node dist/index.js
```

This runs the compiled JS file and shows the output in terminal.

### 6. Output

```
$ node dist/index.js
Hello, TypeScript!"
```

You will see your message in the terminal.

### Important Points

- Make sure Node.js and npm are installed.
- Use npx tsc to compile (no global install needed).
- By default, output goes to dist/ folder.
- You can run tsc --watch for auto compilation.
- If you get errors, check your tsconfig.json and file path.

### Common Mistakes

- Running node src/index.ts (TypeScript file directly).
- Forgetting to run tsc.
- Not creating tsconfig.json (causes config issues).

> "Your first TypeScript program is just the beginning! Keep coding!" ☺

# TYPESCRIPT

## CORE TYPES IN TYPESCRIPT

Strong types = Better code!

### Page 5 : Number, String, Boolean

In TypeScript, we have some core (basic) types that we use all the time.
The most important ones are: `number` , `string` , `boolean`

## 1. number

- The number type is used for all numeric values.
- It can be an integer, decimal, or even special values (like Infinity, NaN).
- TypeScript doesn't have separate types for int, float, etc. – everything is number.

```
// Examples of number type
let age: number = 25;
let price: number = 99.99;
let total: number = -10;
let score: number = Infinity;
let notANumber: number = NaN;
```

All these values are valid for the number type.

**Quick Note**

- number can also be used for floating point values (decimals).
- NaN means "Not a Number".
- Infinity is a special numeric value (when a number is too large).

**Example (number)**

```
let temperature: number = 36.6;
let distance: number = 1000;
let isValid: boolean = true;  // (just showing different type)
```

## 2. string

- The string type is used for text values.
- It represents a sequence of characters.
- You can use single quotes (') or double quotes (") for strings.

```
let name: string = "Muhammad";
let message: string = 'Hello, TypeScript!';
let city: string = "Karachi";
let country: string = 'Pakistan";
```

Both single and double quotes are valid.

**Comparison**

| Feature | number | string | boolean |
|---|---|---|---|
| Used for | Numbers | Text | True/False |
| Example | 25 | "Hello" | true |
| Quotes | Not needed | " or ' | Not needed |

## 3. boolean

- The boolean type is used for true or false values.
- It is often used in conditions, flags, and logic.

```
let isActive: boolean = true;
let isAdmin: boolean = false;
let hasAccess: boolean = true;
```

Only two values are allowed: true or false

**Important**

- TypeScript helps catch type errors at compile time, so you don't accidentally assign the wrong type.
- Always use the correct type for better code readability and fewer bugs.

```
let flag: boolean = true;
```

> "Using the right core types in TypeScript makes your code safer, cleaner and easier to maintain!"

# TYPESCRIPT
# ARRAY, OBJECT, TUPLE

Different types, different uses!

## Page 6 : Array, Object, Tuple

Intermediate Level

## 1. Array

- An array is a collection of values of the same type (or a union of types).
- In TypeScript, you can define the type using [ ] (array syntax) or Array<type> (generic syntax).
- Arrays are zero-based (index starts from 0).

```
// Array of numbers
let numbers: number[] = [1, 2, 3, 4];

// Generic array syntax
let names: Array<string> = ["Ali", "Ahmed", "Sara"];
```

Both ways are correct. Use whichever is more readable for you.

[1, 2, 3]
[ ... ]

### Quick Note

- All elements in the array should be of the same type.
- You can also use union types like (number | string)[ ].
- The array type can be used with other types (e.g. object[], string[ ], etc.).

### Example (Array)

```
let fruits: string[] = ["Apple", "Mango", "Banana"];
let scores: number[] = [10, 20, 30];
fruits.push("Orange");
console.log(fruits); // ["Apple","Mango", "Banana","Orange"]
```

## 2. Object

- An object is used to store multiple values as key-value pairs.
- In TypeScript, you define the structure using type or interface (or inline type).
- Each property can have a specific type.

```
// Object type using type
type User = {
  id: number;
  name: string;
  isActive: boolean;
};
let user: User = {
  id: 1,
  name: "Ali",
  isActive: true
};
```

Objects help you keep your data organized and safe with proper types.

{...}

### Comparison

| Feature | Array | Object |
|---|---|---|
| Structure | List of values | Key-value pairs |
| Syntax | type[] / Array<type> | { key: type } |
| Access | index (0, 1, 2, ...) | key (name, id, ...) |
| Example | [1, 2, 3] | { id: 1, name: "Ali" } |

### Important

- Use objects when you need to store related data.
- Use arrays when you have a list of similar values.
- You can also use nested objects and arrays.

## 3. Tuple

- A tuple is a special type of array with a fixed number of elements, where each element can have a different type.
- Tuples are useful when the order and type of values are important.

```
// Tuple type
let person: [string, number, boolean] = ["Ali", 25, true];
console.log(person[0]); // Ali
console.log(person[1]); // 25
console.log(person[2]); // true
```

The length and type of each element is fixed.

["Ali", 25, true]
↑ string ↑ number ↑ boolean

### Example (Tuple)

```
let coordinates: [number, number] = [10, 20];
let user: [string, number] = ["Hassan", 30];
console.log(coordinates[0]); // 10
```

### Quick Note

- Tuples are less common than arrays.
- They are mostly used in specific cases (like function returns, fixed data, etc.).

> "Choose the right type for your data, it makes your code safer and more predictable!"

# TYPESCRIPT
# ENUM, ANY, UNKNOWN

More types, more control!

Page 7 : Enum, Any, Unknown

Intermediate Level

## 1. Enum

- Enum (short for enumeration) is used to define
- a set of named constant values.
- It makes code more readable and maintainable.
- Enum values can be numeric (default) or string.

```
// Numeric enum (default values)
enum Role {
   Admin,     // 0
   User,      // 1
   Guest      // 2
}

// String enum
enum Direction {
   North = "N",
   South = "S",
   East = "E",
   West = "W"
}
```

By default, numeric enums start from 0 and increase by 1.

You can also assign string values to enums.

### Quick Note

- enum is used for predefined constant values.
- any disables type checking.
- unknown is a safer alternative to any.

### Example (Enum)

```
enum Status {
   Pending = "P",
   Approved = "A",
   Rejected = "R"
}
let orderStatus: Status = Status.Pending;
console.log(orderStatus); // P
```

### Comparison

| Feature | Enum | Any | Unknown |
|---|---|---|---|
| Purpose | Named constants | Disable type checking | Safe unknown type |
| Type Safety | High | None | High (with narrowing) |
| Example | enum Role { } | let a: any | let u: unknown |

## 2. Any

- any is a special type that tells TypeScript to
- ignore type checking.
- It can hold any type of value.
- It should be used only when necessary, as it can lead to runtime errors.

```
let value: any;
value = 10;
value = "Hello";
value = true;
value = { name: "Ali" };
console.log(value); // Works, no error
```

No type checking is done with any.

### Example (Any)

```
function logValue(val: any) {
   console.log(val);
}
logValue(123);
logValue("TypeScript");
logValue({ id: 1 });   // All are valid
```

## 3. Unknown

- unknown is a safer alternative to any.
- It means the value can be of any type, but you must narrow it down before using it.
- It forces you to do type checking (using typeof, instaneoof, etc.).

```
let value: unknown;
value = 10;
value = "Hello";
value = true;
value = { name: "Ali" };
found
// You must check the type before using it.
```

Type checking is required before using the value.

### Example (Unknown)

```
function handleValue(val: unknown) {
   if (typeof val === "string") {
      console.log(val.toUpperCase());
   } else if (typeof val === "number") {
      console.log(val.toFixed(2));
   } else {
      console.log("Unknown type");
   }
}
```

### Quick Note

- Use enum for constant values.
- Use any only when you really need it.
- Use unknown for safer type handling.

> "Use the right type, at the right time, for safer and cleaner code!"

# TYPESCRIPT
## VOID, NULL, NEVER, UNDEFINED

Special types for better control!

### Page 8 : Special Types

Intermediate Level

## ★ 1. void

- void means a function does not return any value.
- It is mainly used for functions that perform an action (like logging, updating, etc.).
- You cannot assign a value to a variable of type void.

```
function showMessage(): void {
  console.log("Hello, TypeScript!");
}

let result: void = showMessage();  // OK
let x: void = 10;     // Error
```

Function returns nothing, so its type is void.

### Quick Note

- void is not the same as undefined or null.
- void is used for function return types, not for variables (in most cases).
- You can only assign undefined or null to a void type (not a value).

### Example (void)

```
function logMessage(): void {
  console.log("This is a void function");
}
logMessage();  // OK
```

## ★ 2. null

- null means no value is present.
- It is often used to clear a variable or indicate that an object is empty.
- TypeScript treats null as a valid type, but you may need to enable strict null checks.

```
let user: string | null = null;
let name: string = null; // Error (without strict null checks)
let name: string | null = null; // OK
```

Use union type with null when needed.

### Comparison

| Feature | void | null |
|---|---|---|
| Meaning | no return value | no value |
| Used in | functions | variables / values |
| Can be assigned | only undefined (or null) | only null |
| Example | let x: void | let x: null |

## ★ 3. never

- never means a value will never occur.
- It is used for functions that always throw an error or never return (e.g. infinite loop).
- It is also useful in exhaustive checks (like switch).

```
function throwError(message: string): never {
  throw new Error(message);
}
function loop(): never {
  while (true) {}  // never returns
}
```

Used when a value can never exist.

### Example (never)

```
function fail(message: string): never {
  throw new Error(message);
}
fail("Something went wrong!");
```

### Important

- never is a subtype of every type.
- It helps in type safety and better code analysis.
- It is often used in error handling and infinite loops.

## ★ 4. undefined

- undefined means a variable is declared but not assigned a value.
- It is the default value of uninitialized variables.
- It is different from null, but both are considered "empty" values.

```
let data: string | undefined;
console.log(data);  // undefined
```

Use union type with undefined when needed.

### Quick Note

- undefined is the default value for variables (if not initialized).
- null is a value you can explicitly assign.
- Both can be used in union types.

> "Use the right type for the right situation, it makes your code safer and more predictable!"

# TYPESCRIPT BIGINT

Used for very large integers!

## Page 9 : BigInt

## 1. BigInt

- bigint is a special type in TypeScript (and JavaScript) that is used to represent integers with unlimited precision.
- It is used when the number is too large to be safely handled by the normal number type (which is limited to $2^{53}-1$).
- bigint values are created by appending n at the end of a number.

### Quick Note

- bigint can handle integers of any size (no fixed limit).
- It is not interchangeable with number.
- You cannot mix bigint and number directly in operations (it throws an error).

```
// Example: bigint
let bigNumber: bigint = 9007199254740991n;
console.log(bigNumber);

// Output:
// 9007199254740991n
```

Use n at the end to make it a bigint.

9007199254740991n

### Example (BigInt)

```
let huge: bigint = 12345678901234567890123456789n;
let another: bigint = BigInt(10);
console.log(huge + another);
```

### Comparison

| Feature | bigint | number |
|---|---|---|
| Purpose | Very large integers | Normal numbers |
| Precision | Unlimited | Up to $2^{53}-1$ |
| Syntax | 123n | 123 |
| Mixing | Not allowed | Not allowed (with bigint) |

### Important

- Use bigint when working with very large integers (e.g. IDs, timestamps, cryptography, etc.).
- Always end the value with n.
- It is a separate type and cannot be mixed with number.

## Summary of All Core Types In TypeScript

| Type | What it is | Example | Key Points |
|---|---|---|---|
| number | Represents numeric values (integers & decimals). | let x: number = 42; | Used for most numeric operations. (0, 1, -5, 3,14, etc.). |
| string | Represents text values. | let name: string = "Ali"; | Surrounded by '' or "". |
| boolean | Represents true or false. | let isActive: boolean = true; | Only two values: true / false. |
| array | Stores multiple values of the same type. | let nums: number[] = [1, 2, 3]; | Use [] or Array<type>. |
| object | Stores key-value pairs. | let user: {name: string} = {name:"Ali"}; | Uses { } and properties. |
| tuple | Fixed number of elements (with specific types). | let person: [string, number] = ["Ali", 25]; | Order and types are fixed. |
| enum | Named set of constant values. | enum Role { Admin, User } | Useful for predefined values. |
| any | Disables type checking. | let data: any = "text"; | Use only when necessary. |
| unknown | Safer alternative to any. | let data: unknown = value; | Requires type checking before use. |
| void | No value is returned. | function log(): void {} | Used for functions that don't return a value. |
| null | Represents no value. | let data: null = null; | Explicitly means no value. |
| never | Represents a value that never occurs. | function error(): never { throw new Error(); } | Used in unreachable code (e.g. throw). |
| undefined | Value is not assigned. | let data: undefined; | Common when variable is not initialized. |
| bigint | For very large integers. | let big: bigint = 1234567890123456789n; | Unlimited precision for integers. |

> "TypeScript gives you the right type, so your code is safer, smarter and easier to maintain!"

# TYPESCRIPT
# TYPE INFERENCE & TYPE ASSERTIONS

TypeScript helps you write safer code!

Page 10 : Type Inference & Type Assertions

## ★ 1. Type Inference

- Type inference means TypeScript automatically guesses the type of a variable based on the value you assign to it.
- You don't have to explicitly write the type (in many cases).
- It keeps your code clean and reduces extra typing.

```
// TypeScript infers the type as 'number'
let age = 25;

// TypeScript infers the type as 'string'
let name = "Ali";

// TypeScript infers the type as 'boolean'
let isActive = true;
```

Here, TypeScript automatically knows the type of each variable.

### Quick Note

- Type inference works best when you assign a value at the time of declaration.
- If you don't assign a value, TypeScript may infer the type as any. (or give an error in some cases).

### Example – Inference with Arrays

```
let numbers = [1, 2, 3];
// Inferred as number[]

let names = ["Ali", "Ahmed"];
// Inferred as string[]
```

TypeScript figures out the array element type automatically.

### Important

- Type inference is not magic – it follows simple rules.
- It only works when it can clearly determine the type.
- You can still use explicit types when needed.

## ★ 2. Type Assertions

- Type assertions tell TypeScript .to treat a value as a specific type (even if it cannot be automatically determined).
- They are used when you are sure about the type, but TypeScript is not.
- Type assertions do not change the actual value at runtime – they only affect the type checking.

```
// Using 'as' syntax
let data: unknown = "Hello";
let message = data as string;  // OK

// Using angle-bracket syntax
let user = <string>data;  // OK
```

You are telling TypeScript: "Trust me, this is a string!"

### Example – Type Assertion

```
let value: unknown = "123";
let strLength: number = (value as string).length;
// TypeScript allows it (but be careful!)
```

Here we asserted that value is a string.

### When to Use?

- When you get data from an API {(unknown type).
- When working with DOM elements (e.g. getElementById).
- When you are sure about the type, but TypeScript cannot infer it.

### Caution

- Use type assertions carefully.
- If you are wrong about the type, it can cause runtime errors.
- It bypasses some of TypeScript's safety checks.

> "Type Inference saves you typing,
> Type Assertions give you control!"

# TYPESCRIPT UNION TYPE & TYPE NARROWING

More power with better type safety!

Page 11 : Union Type & Type Narrowing

## 1. Union Type

- A union type allows a variable to be one of multiple types.
- It is written using the | (pipe) symbol.
- It is useful when a value can be of different types (e.g. string or number).
- At runtime, you need to narrow the type before using type-specific properties or methods.

```
let id: string | number;
id = 123;    // OK
id = "abc";  // OK
id = true;   // Error
```

Only string or number are allowed, not boolean.

### Quick Note

- You can use union types with primitives, objects, arrays, etc.
- It's one of the most used types in TypeScript.

### Example – Union type with function

```
function getValue(): string | number {
  return Math.random() > 0.5 ? "Hello" : 42;
}

const val = getValue();
console.log(val);  // string or number
```

The return type can be either string or number.

### Important

- Use union types when a variable can be one of several types.
- You should narrow the type before accessing specific properties.

## 2. Type Narrowing

- Type narrowing is the process of telling TypeScript the exact type of a variable within a certain scope.
- It helps you safely access properties and methods that are specific to a type.
- It can be done using typeof checks, instanceof, truthy checks, or custom type guards.

### Example 1 – typeof check

```
function printId(id: string | number) {
  if (typeof id === "string") {
    console.log(id.toUpperCase());  // string method
  } else {
    console.log(id.toFixed(2));     // number method
  }
}
```

typeof helps TypeScript know the exact type.

### Example 2 – instanceof check

```
class Cat {
  meow() { console.log("Meow!"); }
}
class Dog {
  bark() { console.log("Woof!"); }
}

function animalSound(animal: Cat | Dog) {
  if (animal instanceof Cat) {
    animal.meow();  // Cat methods
  } else {
    animal.bark();  // Dog methods
  }
}
```

instanceof checks the actual class type.

### Example 3 – in operator (for objects)

```
type User = { name: string; age: number };
type Admin = { role: string };

function getRole(user: User | Admin) {
  if ("role" in user) {
    console.log(user.role);  // Admin
  } else {
    console.log(user.name);  // User
  }
}
```

The in operator checks for property existence.

### Quick Note

- Type narrowing helps with safer code and better autocomplete in editors.
- Common methods: typeof, instanceof, in, truthy check, user-defined type guards.

> "Narrow the type, unlock the power of TypeScript!"

# TYPESCRIPT INTERFACE

Page 12 : Interface

Intermediate Level

Interfaces help in building clean & structured code!

## 1. What is Interface?

- An interface is a way to define the structure of an object in TypeScript.
- It tells TypeScript what properties and methods an object should have.
- It is used for type-checking and better code organization.
- Interfaces are only used during development (type checking). They don't exist in the compiled JS.

```
// Example of a simple interface
interface User {
  name: string;
  age: number;
}
```

This defines the structure of an object with name and age properties.

### Quick Note

- Interfaces are used to define the shape of an object (or class).
- They help in writing clean, reusable and maintainable code.
- They are mainly used for objects, classes and functions (with parameters or return types).

### Example – Interface in Action

```
interface User {
  name: string;
  age: number;
}
const user: User = {
  name: "Ali",
  age: 25
};
```

This object follows the User interface, so TypeScript will not show any error.

## 2. Basic Example

- We can use interfaces to define the structure of an object and then use it to type a variable, parameter or return value.
- This makes the code more readable and reduces mistakes.

```
interface Product {
  name: string;
  price: number;
  inStock: boolean;
}
const item: Product = {
  name: "Laptop",
  price: 1200,
  inStock: true
};
```

Here, the object "item" follows the Product interface.

### Why Use Interfaces?

| Without Interface | With Interface |
|---|---|
| No type checking | Type checking is enabled |
| Harder to maintain | Cleaner and organized code |
| Less readable | More readable and self-explanatory |
| Can lead to errors | Reduces errors |

### Important

- Interfaces are not used to create instances.
- They are only for defining the structure.
- You can extend interfaces (we will see this later).

## 3. Optional Property

- Sometimes, a property in an interface is not mandatory.
- We can use ? (question mark) to make a property optional.
- This means the object can have that property or not.

```
interface User {
  name: string;
  age: number;
  bio?: string;  // optional property
}
```

bio is optional. So, the object can have it or it can be missing.

### Example – Optional Property

```
const user1: User = {
  name: "Ahmed",
  age: 20
};
// Also valid (bio is optional)
const user2: User = {
  name: "Sara",
  age: 22,
  bio: "Developer"
};
```

Both are valid because bio is optional.

"Interfaces make your code safer, cleaner and easier to maintain!"

# TYPESCRIPT INTERFACE

Page 13 : Interface (Advanced)

Interfaces make your code more readable & powerful!

## 1. Readonly Property

Intermediate Level

- The readonly keyword is used to make a property read-only in an interface.
- It prevents the property from being changed after initialization.
- It is useful when you want to keep the value constant (e.g. id, name, etc).

```ts
// Example: readonly property in interface
interface User {
  readonly id: number;
  name: string;
}
const user: User = {
  id: 101,
  name: "Ali"
};
// user.id = 202;  // Error: Cannot assign to 'id'
```

The id property is readonly, so it cannot be changed after initialization.

readonly

### Comparison

| Property | Normal | Readonly |
|---|---|---|
| Reassign allowed? | Yes | No |
| Used for | Mutable data | Constant data |
| Example | name: string | readonly id: number |

### Important

- readonly is shallow (doesn't make nested objects readonly).
- It only stops direct reassignment of the property.
- You can still modify object properties inside (unless they are also readonly).

### Quick Note

- readonly can be used with any property in an interface.
- It only prevents reassignment, not the value itself (if it's an object, its internal values can still change).

### Example – Readonly with Object

```ts
interface User {
  readonly profile: {
    name: string;
  };
}
const user: User = {
  profile: { name: "Ali" }
};
// user.profile.name = "Ahmed";  // ✔ Allowed
```

Here, profile is readonly, but you can still change the name inside (because profile itself is an object).

## 2. Function Inside Interface

- Interfaces can also define function types.
- This allows you to specify the parameters and return type of a function.
- It helps to ensure that objects follow a specific method signature.

```ts
// Example: interface with function
interface Calculator {
  add(a: number, b: number): number;
  subtract(a: number, b: number): number;
}
const calc: Calculator = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b
};
```

Functions in interfaces are just like methods in classes.

(a, b) => number

### Example – Multiple Functions

```ts
interface UserService {
  getUser(id: number): string;
  updateUser(id: number, name: string): boolean;
}
const service: UserService = {
  getUser: (id) => "User " + id,
  updateUser: (id, name) => true
};
```

Multiple methods can be defined inside one interface.

### Important

- The function signature must match exactly.
- You can use optional parameters and union types in function definitions too.

### Quick Note

- You can define multiple functions in one interface.
- The function name, parameters and return type must match.

## 3. Interface With Arrays

- Interfaces can be used to type arrays of objects.
- This helows in defining the structure of each object inside the array.
- It makes your code more organized and type-safe.

```ts
// Example: interface with array
interface Product {
  id: number;
  name: string;
  price: number;
}
const products: Product[] = [
  { id: 1, name: "Laptop", price: 1200 },
  { id: 2, name: "Mouse", price: 25 }
];
```

Product[] means an array of Product objects.

{}, {}

### Example – Array of Interfaces

```ts
interface Student {
  id: number;
  name: string;
  subjects: string[];
}
const students: Student[] = [
  { id: 1, name: "Ali", subjects: ["TS", "JS"] },
  { id: 2, name: "Sara", subjects: ["React", "Node"] },
];
```

Each object in the array must follow the Student interface.

### Example – Array of Objects (Simple)

```ts
interface Todo {
  id: number;
  title: string;
  completed: boolean;
}
const todos: Todo[] = [];  // array of Todo objects
```

### Quick Note

- You can also use readonly with arrays (e.g. readonly Product[]).
- Arrays can store multiple objects if they follow the interface structure.

### Important

- Use interface with arrays when you have a list of objects with the same structure.
- It helps in validation, clean code and better maintainability.

> "Interfaces keep your data organized, your code clean and your projects scalable!"

# TYPESCRIPT INTERFACE

## Page 14 : Extending Interface

Intermediate Level

Interfaces can extend other interfaces!

## 1. What is Extending Interface?

- Extending an interface means creating a new interface that inherits (gets) the properties and methods of an existing interface.
- It helps in reusing code and building a more specific or advanced structure.
- You can extend multiple interfaces as well (using comma).

### Quick Note

- The child interface will have all the members of the parent interface(s).
- You can also add new properties or methods in the child interface.

## 2. Extending Multiple Interfaces

- An interface can extend more than one interface by using a comma (,).
- It will get all the properties and methods from all the extended interfaces.

```
// Two separate interfaces
interface A {
  a: string;
}

interface B {
  b: number;
}

// Extending both A and B
interface C extends A, B {
  c: boolean;
}
```

Interface C now has properties: a, b and c.

A + B = C

## 3. Extending with New Members

- You can override existing members (with the same type) or add new members in the child interface.
- This is useful for creating specialized interfaces.

```
interface User {
  id: number;
  name: string;
}
interface Admin extends User {
  role: "admin" | "moderator";
  permissions: string[];
  // new members
}
```

Admin gets id and name from User, and adds role and permissions.

## Example 1 - Simple Extension

```
// Base interface
interface Person {
  name: string;
  age: number;
}

// Extending Person
interface Employee extends Person {
  employeeId: number;
  department: string;
}
```

Employee inherits name and age from Person, and adds its own properties.

## Example 1 - Usage

```
const emp: Employee = {
  name: "Ali",
  age: 25,
  employeeId: 101,
  department: "IT"
};
```

Now emp has all properties of Person + its own properties.

## Example 2 - Multiple Extension

```
interface Animal {
  name: string;
}
interface Flyable {
  fly(): void;
}
interface Bird extends Animal, Flyable {
  color: string;
}
```

Bird extends two interfaces - Animal and Flyable.

## Example 2 - Usage

```
const bird: Bird = {
  name: "Parrot",
  color: "Green",
  fly: () => console.log("Flying...")
};
```

bird has name, color (from Bird) and fly() (from Flyable).

### Quick Notes

- You can extend multiple interfaces at once.
- The child interface inherits all members of parent interfaces.
- You can add new properties or methods in the child interface.
- It's a powerful way to keep your code clean, reusable and well-structured.

"Extend, Reuse, Build Better with Interfaces!"

# TYPESCRIPT TYPE ALIAS

Page 15 : Type Alias

Type System Feature

Type Alias lets us create custom names for types.

## 1. What is Type Alias?

- A Type Alias is a way to give a custom name to an existing type in TypeScript.
- It helps make your code more readable, reusable and maintainable.
- Type aliases are created using the keyword "type" followed by a name, and then the type you want to alias.
- It can be used for primitive types, union types, object types, tuples, etc.

### Quick Note

- Type alias is only a compile-time feature.
- It does not exist in the final JavaScript code (after compilation).

## 2. Function Type

- Type aliases are also very useful for defining function types.
- We can create a type alias for a function signature (parameters and return type).

```
type GreetFunction = (name: string) => string;

const greet: GreetFunction = (name) => {
   return `Hello, ${name}!`;
};

console.log(greet("Ali"));  // Hello, Ali!
```

Here, GreetFunction is a type alias for a function that takes a string and returns a string.

### Quick Notes

- Function type aliases make function definitions shorter and cleaner.
- You can also use them with multiple parameters.
- They are especially useful when you need to reuse the same function signature in different places.

## Example 1 - Basic Example

```
// Create a type alias for a string
type UserName = string;

let name: UserName = "Hussain";

console.log(name);   // Output: Hussain
```

Here, UserName is a custom name for the string type.

### Usage

- We created a type alias UserName for string;
- Now we can use UserName anywhere a string is expected.
- It makes the code cleaner and easier to update if the type changes later.

Instead of using string everywhere, we use a name!

## Example 2 - Function Type with Multiple Params

```
type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (a, b) => a + b;

const multiply: MathOperation = (a, b) => a * b;

console.log(add(5, 3));       // 8
console.log(multiply(5, 3));  // 15
```

### Usage

- MathOperation is a type alias for a function.
- It takes two numbers and returns a number.
- We can use the same type alias for multiple functions like add and multiply.

Same type alias, multiple functions!

## Example 3 - Union with Function Type

```
type IDType = string | number;
type GetId = (id: IDType) => void;

const getId: GetId = (id) => {
   console.log(`ID: ${id}`);
};

getId("abc");   // ID: abc
getid(123);     // ID: 123
```

Type alias can also be used with union types in function parameters.

"Type Alias makes your TypeScript code cleaner, reusable and easier to manage!"

# TYPESCRIPT TYPE ALIAS



## Page 16 : Intersection & Literal Type

Type System Feature

Type Alias helps us create custom names for types.

## 1. Intersection Type

- Intersection type ( & ) allows us to combine multiple types into one.
- The resulting type will have all the properties of the combined types.
- It is useful when a value should satisfy more than one type at the same time.

### Quick Note

- Use & (AND) to combine types.
- The value must have all the properties from both (or all) types.

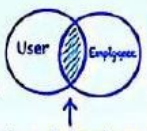
### Example - Intersection Type

```ts
// Create two types
type User = {
  name: string;
  age: number;
};

type Employee = {
  employeeId: number;
  department: string;
};

// Combine them using intersection type
type UserEmployee = User & Employee;

// Example object
const person: UserEmployee = {
  name: "Ali",
  age: 25,
  employeeId: 101,
  department: "IT"
};
```

The resulting type must have all properties from both User and Employee.

### Usage

- When you need a type that combines multiple existing types.
- Useful for objects that have more than one set of properties.




User & Employee

## 2. Literal Type

- Literal type allows us to specify exact values that a variable can have.
- It can be a string, number, boolean, or even a union of literal values.
- It helps in creating more specific and safer types.

### Quick Note

- Use literal types when you want to restrict the value to a specific set of values. It makes your code more predictable and safer.

### Example - Literal Type

```ts
// Single literal type (string)
type Status = "success" | "error" | "loading";
let currentStatus: Status = "success";
// currentStatus = "failed"; // Error
```

Only these values are allowed.

```ts
// Number literal type
type Direction = 0 | 1 | 2 | 3;
let move: Direction = 1;
// move = 5; // Error
```

Only 0, 1, 2 or 3 are allowed.

```ts
// Boolean literal type
type YesNo = true | false;
let answer: YesNo = true;
// answer = "yes"; // Error
```

Only true or false are allowed.

```ts
// Using literal type with object
type Role = "admin" | "user" | "guest";
type User = {
  name: string;
  role: Role;
};
const u: User = { name: "Hussain", role: "admin" };
```

Role can only be one of the given values.

### Usage

- Use literal types to restrict values.
- Commonly used in APIs, enums, and configuration objects.
- Works great with union types for multiple values.

> "Type Aliases + Intersection + Literal Types = More Safe & Maintainable Code!"

# TYPESCRIPT TYPE ALIAS

## Page 17 : Advanced Type Alias

Type System Feature

Type Alias helps us create custom names for types.

## 1. Array Of Object

- An array of objects means a type alias for an array where each element is an object with specific properties.
- It helps to keep your code clean and reusable.
- Useful when working with lists of data (e.g. users, products, etc.).

### Quick Notes

- Use [] to define an array type.
- Type alias can be used with array of objects for better readability.

### Example 1 - Array Of Object

```ts
// Create a type alias for an array of objects
type User = {
  id: number;
  name: string;
  email: string;
};

type Users = User[];
// Array of User objects

const users: Users = [
  { id: 1, name: "Ali", email: "ali@example.com" },
  { id: 2, name: "Sara", email: "sara@example.com" }
];
```

Here, Users is a type alias for an array of User objects.

### Usage

- Use it for lists of objects like users, products, orders, etc.
- Makes the code more readable and reusable.

## 2. Optional Property

- An optional property means a property that may or may not exist in an object.
- It is defined using ? after the property name.
- It is useful when some data is not always available.

### Quick Notes

- Use ? to make a property optional.
- The type of the property stays the same.
- It helps in creating flexible and reusable types.

### Example 2 - Optional Property

```ts
type User = {
  id: number;
  name: string;
  email?: string;  // optional property
};

const user1: User = {
  id: 1,
  name: "Ali"
};

const user2: User = {
  id: 2,
  name: "Sara",
  email: "sara@example.com"
};
```

email is optional, so it can be present or absent.

### Usage

- Use it when a field is not always required.
- Common in forms, API responses, and database models.

## 3. Readonly Property

- A readonly property means the value of a property cannot be changed after it is assigned.
- It helps to protect data and prevent accidental modifications.
- Useful when you want to keep an object immutable (at least at the type level).

### Quick Notes

- Use readonly before the property name.
- It only affects the type system (not runtime).
- Great for constant data and immutable objects.

### Example 3 - Readonly Property

```ts
type Product = {
  readonly id: number;
  name: string;
  price: number;
};

const product: Product = {
  id: 101,
  name: "Laptop",
  price: 999
};

// ✗ This will give an error
product.id = 102;
// Cannot assign to 'id' because it is a readonly property.
```

id is readonly, so we can't change it after assigning the value.

### Usage

- Use it for values that should not be changed.
- Helpful for constants, IDs, and fixed data.

> "Type Alias gives you flexibility, cleaner code and better type safety!"

# TYPESCRIPT CLASSES OOP IN TS

## Page 18 : Classes OOP In TS

Classes help us create blueprints for objects in TS.

### Class Basic Syntax

## 1. What is a Class?

- A class is a blueprint for creating objects.
- It helps us bundle data (properties) and behavior (methods) together.
- TypeScript supports OOP (Object-Oriented Programming) using classes, just like other modern languages.
- Classes make code more organized, reusable and maintainable.

### Quick Notes

- A class is defined using the class keyword.
- The constructor is used to initialize properties.
- You can create multiple objects (instances) from a class.

## 2. Class Basic Syntax

- The basic structure of a class in TypeScript:

```
class ClassName {
   // properties (data)
   property: type;

   constructor(value: type) {
      this.property = value;
   }

   // methods (behavior)
   methodName(): void {
     console.log("Hello from Class");
   }
}
```

Class keyword is used to define a class.

Constructor initializes the properties.

Class = Blueprint + Behavior

## Example - Basic Class

```
class Person {
  name: string;
  age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  greet(): void {
    console.log(`Hello, my name is ${this.name}`)
    + ` and I am ${this.age} years old.`);
  }
}

const p = new Person("Ali", 25);
p.greet();   // Hello, my name is Ali and I am 25 years old.
```

We created a class with two properties and one method.

### Using the Class

```
const person1 = new Person("Sara", 20);
const person2 = new Person("Hussain", 22);

person1.greet();   // Hello, my name is Sara...
person2.greet();   // Hello, my name is Hussain..
```

We can create multiple objects from the same class.

## Example - Class with Methods

```
class Car {
  brand: string;
  model: string;

  constructor(brand: string, model: string) {
    this.brand = brand;
    this.model = model;
  }

  start(): void {
    console.log(`The ${this.brand} ${this.model}
    is starting...`);
  }

  stop(): void {
    console.log(`The ${this.brand} ${this.model}
    is stopped...`);
  }
}

const car = new Car("Toyota", "Corolla");
car.start();   // The Toyota Corolla is starting...
car.stop();    // The Toyota Corolla is stopped...
```

Methods are functions inside a class that work with the class data.

### Key Points to Remember

- class keyword is used to create a class.
- constructor() initializes properties.
- this keyword refers to the current instance.
- We can create multiple objects from a class.
- Classes help in code organization and reusability.

### Quick Recap

- Class = blueprint
- Constructor = initialize
- Properties = data
- Methods = behavior
- new = create object

Classes make your code clean, organized and reusable!

> "Classes in TypeScript turn your code into real-world models!"

# TYPESCRIPT CLASSES OOP IN TS

## Page 19 : Access Modifiers with Examples

Class & Access Modifiers

Access modifiers control the visibility of class members (properties & methods).

### 1. What are Access Modifiers?

- Access modifiers are keywords used in classes to control the visibility of class members (properties, methods, and constructors).
- They help with encapsulation and data hiding.
- TypeScript supports three access modifiers:

public | private | protected

**Quick Notes**

- If no modifier is used, it is public by default.
- Access modifiers work with classes and inheritance.
- They do not work with interfaces.

### 2. public Access Modifier

- Members marked as public are accessible from anywhere (inside or outside the class).
- It is the default access modifier.

**Example – public**

```
class Person {
  public name: string;
  public age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  public getDetails(): void {
    console.log(`Name: ${this.name}, Age: ${this.age}`);
  }
}

const p = new Person("Ali", 25);
p.getDetails();    // Accessible from outside
```

public members can be accessed from anywhere (including outside the class).

**Quick Notes**

- Use public for data that needs to be accessed by other parts of your application.
- It is the most commonly used modifier.

### 3. private Access Modifier

- Members marked as private are only accessible within the same class.
- They cannot be accessed from outside the class or from a child class.

**Example – private**

```
class Person {
  private name: string;
  private age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  private getInfo(): string {
    return `${this.name} - ${this.age}`;
  }
}
const p = new Person("Sara", 22);
console.log(p.getInfo());   // ✔ OK (inside class)
console.log(p.name);        // ✖ Error (private)
```

private members are only accessible inside the class.

### 4. protected Access Modifier

- Members marked as protected are accessible within the class and its child classes (but not from outside).

**Example – protected**

```
class Animal {
  protected name: string;

  constructor(name: string) {
    this.name = name;
  }
  protected showName(): void {
    console.log(`Name: ${this.name}`);
  }
}
class Dog extends Animal {
  constructor(name: string) {
    super(name);
  }
  display(): void {
    this.showName();   // ✔ Accessible (in child class)
  }
}
const dog = new Dog("Buddy");
dog.display();     // // Works
console.log(dog.name);   // ✖ Error (outside class)
```

protected members are accessible in the class and child classes.

### Comparison of Access Modifiers

| Modifier | Accessible In | Default? |
|---|---|---|
| public | Anywhere (any class, object) | Yes |
| private | Only within the same class | No |
| protected | Within class & child classes | No |

### Usage

- Use private for sensitive data (e.g. passwords).
- Use protected when you want to allow access in child classes (e.g. base class data).
- Use public for data/methods that need to be accessible everywhere.

> "Access Modifiers give you control and keep your code safe & clean!"

# TYPESCRIPT CLASSES OOP IN TS

## Page 20 : Inheritance

Class & OOP Concept

Inheritance lets you reuse code and build better class hierarchies!

## 1. What is Inheritance?

- Inheritance is a feature of Object-Oriented Programming (OOP) that allows a class to inherit properties and methods from another class.
- It helps to reuse code and create a hierarchical structure of classes.
- The child class (subclass) can also add its own properties and methods.

### Quick Notes

- The class that gets properties from another class is called a child class (subclass).
- The class whose properties are inherited is called a parent class (superclass).
- Use the keyword **extends** to inherit a class.

## 2. Basic Inheritance Example

```
// Parent class (Super class)
class Animal {
   name: string;
   constructor(name: string) {
      this.name = name;
   }

   makeSound(): void {
      console.log(this.name + " makes a sound");
   }
}

// Child class (Sub class)
class Dog extends Animal {
   breed: string;
   constructor(name: string, breed: string) {
      super(name); // call parent constructor
      this.breed = breed;
   }

   bark(): void {
      console.log(this.name + " barks!");
   }
}
```

The child class extends the parent class using the extends keyword.

It can use parent class properties and methods.

### Usage

```
const dog = new Dog("Buddy", "Golden Retriever");
dog.makeSound();      // Buddy makes a sound
dog.bark();           // Buddy barks!
```

The child class can access both its own and parent class methods.

## Example 2 - Overriding Method

```
class Animal {
   makeSound(): void {
      console.log("Animal makes a sound");
   }
}

class Cat extends Animal {
   // Override parent class method
   makeSound(): void {
      console.log("Cat meows");
   }
}
```

The child class can override the parent class method with its own implementation.

### Usage

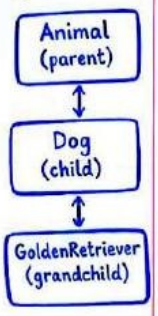
```
const cat = new Cat();
cat.makeSound();      // Cat meows
```

Overriding changes the behavior of the inherited method!

## Example 3 - Multi-Level Inheritance

```
class Animal {
   name: string;
   constructor(name: string) {
      this.name = name;
   }

   makeSound(): void {
      console.log(this.name + " makes a sound");
   }
}

class Dog extends Animal {
   breed: string;
   constructor(name: string, breed: string) {
      super(name);
      this.breed = breed;
   }
}

class GoldenRetriever extends Dog {
   color: string;
   constructor(name: string, breed: string, color: string) {
      super(name, breed);
      this.color = color;
   }

   fetch(): void {
      console.log(this.name + " fetches the ball!");
   }
}
```

A class can inherit from another class, which is already a child class.




### Quick Recap

- Inheritance helps reuse code.
- Use extends to inherit a class.
- The child class can override parent methods.
- You can have multi-level inheritance.
- It helps to build a clean and structured code.

> "Inheritance saves time, reduces code duplication and builds better classes!"

# TYPESCRIPT CLASSES OOP IN TS

## Page 21 : Abstract Class

Class & OOP Concept

Abstract classes cannot be instantiated, they are meant to be extended!

### 1. What is an "Abstract Class?

- An abstract class is a class that cannot be instantiated on its own.
- It is used to provide a base structure for other classes.
- It can contain abstract methods (without implementation) and concrete methods (with implementation).
- Child classes must implement the abstract methods.

#### Quick Notes

- Use abstract when you want to define a common structure for multiple classes.
- Abstract class = blueprint + rule.
- Cannot create object of an abstract class.

### 2. Basic Example

```
// Abstract class with an abstract method
abstract class Shape {
    name: string;
    constructor(name: string, radius: number) {
        this.name = name;
        this.radius = radius;   // initialize radius
    }

    // Abstract method (no implementation)
    abstract makeSound(): void;

    // Concrete method (with implementation)
    move(): void {
        console.log(`${this.name} moves.`);
    }
}

// Child class must implement the abstract method
class Circle extends Shape {
    makeSound(): void {
        console.log("Circle sound!");
    }
}
```

Abstract method has no body. Child class must provide its own implementation.

Animal (abstract) ↓ Dog (extends)

### 3. Abstract Class with Multiple Methods

```
// Abstract class with multiple abstract methods
abstract class Vehicle {
    name: string;
    constructor(name: string) {
        this.name = name;
    }

    abstract start(): void; // abstract method
    abstract stop(): void;  // abstract method

    move(): void {
        console.log(`${this.name} is moving.`);
    }
}

class Car extends Vehicle {
    start(): void {
        console.log("Car started!");
    }

    stop(): void {
        console.log("Car stopped!");
    }
}
```

Both methods are abstract in the base class and must be implemented in the child class.

### 4. Another Child Class (with Fix)

```
class Bike extends Vehicle {
    start(): void {
        console.log("Bike started!");
    }

    stop(): void {
        console.log("Bike stopped!");
    }
}
```

Each child class must implement all abstract methods from the base class.

### Key Points to Remember

- Abstract class cannot be instantiated.
- It can have both abstract and concrete methods.
- Child classes must implement all abstract methods.
- Used to provide a common structure for multiple classes.
- Helps in code organization and maintainability.

### Usage

```
const circle = new Circle("Circle", 10);
circle.move();          // Circle moves.
circle.makeSound();     // Circle sound!
console.log(circle.radius); // 10
```

"You can create objects of the child class, not the abstract class.

### Quick Recap

- Abstract class = template.
- Abstract method = no body (must be implemented).
- Child class provides the actual logic.
- You can't create an object of an abstract class.

> "Abstract classes help you build better and more flexible code structure!"

# TYPESCRIPT GENERICS

Generics make your code flexible, reusable and type-safe!

## Page 22 : Generics

### Generics Concept

## 1. What is Generics?

- Generics allow you to create reusable components (like functions, classes, or interfaces) that can work with different types, instead of a single type.
- It helps you write flexible and type-safe code.
- You can use a placeholder (usually <T>) to represent a type, which will be decided when the code is used.

### Quick Notes

- Generics are a way to write code that works with multiple types.
- The type is decided at the time of using the function, class or interface.
- They help to avoid code duplication.

## 2. Generics with Function

- You can use generics in functions to make them work with any type.
- The type is passed using <T> (where T is a type variable).

```
// Generic function to return the value
function identity<T> (value: T): T {
  return value;
}

// Using the function
let result1 = identity<string>("Hello"); // string
let result2 = identity<number>(123);    // number
```

T is a type variable. It can be any type.

### Quick Note

- The function identity<T> can return any type.
- When we call the function, we specify the type in < > (e.g. <string>, <number>).

## 3. Generics with Arrays in Function

- You can also use generics with arrays in functions.
- This helps to work with arrays of any type, without writing separate functions for each type.

```
// Generic function to get the first element of an array
function getFirstElement<T>(arr: T[]): T {
  return arr[0];
}

// Using the function
let num = getFirstElement<number>([10, 20, 30]);
let str = getFirstElement<string>(["a", "b", "c"]);
```

T[] means an array of type T.

### Example – Generic Array Function

```
function printArray<T>(items: T[]): void {
  items.forEach(item => console.log(item));
}

// Using the function
printArray<number>([1, 2, 3]);
printArray<string>(["a", "b", "c"]);
```

Works with arrays of any type (number, string, etc.).

### Important

- Generics make your functions more reusable.
- You can use generics with arrays, objects, classes and interfaces.
- The type is decided when you call the function.

### Summary

- Generics = reusable + type-safe code.
- Use <T> to define a type variable.
- Works with functions, arrays, classes, interfaces, etc.
- Helps in writing flexible and maintainable code.

> "Generics don't make your code complicated, they make it powerful!"

# TYPESCRIPT GENERICS

## Page 23 : Generics (Part 2)

### Generics with Interface, Type Alias & Classes

Generics make your code more flexible, reusable and powerful!

## 1. Generics with Interface

- You can use generics with interfaces to create flexible and reusable contracts.
- It allows the interface to work with different types instead of a fixed type.
- The type is specified when the interface is used.

**Quick Notes**

- The interface becomes a generic interface.
- You can pass any type when using it.
- It helps in building reusable components.

**Example - Generic Interface**

```
interface Response<T> {
  data: T;
}

function getData<T>(value: T): Response<T> {
  return { data: value };
}
// Usage
const res1 = getData<string>("Hello");
const res2 = getData<number>(123);
```

The interface is generic and can work with different types.

**Important**

- Generic interfaces make your code more flexible and reusable with different types.

**Summary**

- Use generics with interfaces for flexible contracts.
- Specify the type when using it.
- Works with any data type.

## 2. Generics with Type Alias

- You can use generics with type alias to create flexible custom types.
- It helps you define a type that can work with different types.
- Type alias with generics is useful for reusable and cleaner code.

**Quick Notes**

- Type alias can also be generic.
- You can use it for objects, arrays, functions, etc.
- It makes complex types easier to manage.

**Example - Generic Type Alias**

```
type Pair<T> = {
  first: T;
  second: T;
};

const userPair: Pair<string> = {
  first: "Ali",
  second: "Khan"
};

const numberPair: Pair<number> = {
  first: 10,
  second: 20
};
```

Type alias with generics and create custom types with any type.

**Important**

- Type alias with generics helps you create reusable and simplified custom types.

**Summary**

- Use generics with type alias for custom types.
- Can be used with objects, arrays, functions, etc.
- Makes your code cleaner and flexible.

## 3. Generics with Classes

- You can use generics with classes to create flexible and reusable class components.
- It allows the class to work with different types instead of a fixed type.
- Generics in classes are very useful in data structures and real-world applications.

**Quick Notes**

- The class becomes a generic class.
- You can specify the type when creating an object.
- It helps in making the class reusable for multiple types.

**Example - Generic Class**

```
class Box<T> {
  private value: T;

  constructor(val: T) {
    this.value = val;
  }

  getValue(): T {
    return this.value;
  }
}
  const stringBox = new Box<string>("Hello");
  const numberBox = new Box<number>(100);
```

The class works with different types using generics.

**Important**

- Generics in classes help you create reusable and type-safe components.

**Summary**

- Use generics with classes for reusable components.
- Specify the type when creating objects.
- Helps in building scalable applications.

Same Logic
Different Types
More Possibilities!

> "Generics give you the power to write code that works with any type!"

# TYPESCRIPT TYPE GUARD

## Page 24 : Type Guard

### Type Narrowing

Type guards help TypeScript understand the exact type at runtime!

## 1. What is Type Guard?

- A type guard is a way to check the actual type of a value at runtime.
- It helps TypeScript narrow down the type of a variable inside a block of code.
- It is used to safely access properties or methods that exist on a specific type.
- Type guards can be written using typeof, instanceof, user-defined functions, or in-operator.

### Quick Notes

- Type guard = runtime check + type narrowing.
- It helps in working with union types.
- Without type guard, TypeScript may give an error.

## 2. "typeof" Type Guard

- The typeof operator can be used as a type guard to check the primitive type of a value.
- It is useful when a variable can hold multiple types (like string | number | boolean, etc.).
- TypeScript narrows the type based on the typeof check.

```ts
// Example: typeof Type Guard
function getValue(value: string | number) {
  if (typeof value === 'string') {
    return value.toUpperCase();
  } else {
    return value.toFixed(2);
  }
}

const result1 = getValue('hello'); // string
const result2 = getValue(12.5);    // number
```

The typeof operator narrows the type to string or number inside the if-else block.

typeof

### Quick Notes

- typeof returns a string (like 'string', 'number', etc.).
- It is used for primitive types (string, number, boolean, etc.).
- It cannot check complex types (like object, array, etc.).

## Example - Type Guard (Definition in Action)

```ts
type Admin = { name: string; role: 'admin'; }
type User = { name: string; role: 'user'; }
type Account = Admin | User;

function printRole(account: Account) {
  if ('role' in account) {
    // Type guard using in operator
    console.log("Role:", account.role);
  } else {
    console.log("No role found");
  }
}
```

Here, "in" operator helps TypeScript narrow the type to access "role" property.

### Important

- Type guards are used to narrow the type of a variable.
- They make code safer and reduce runtime errors.

## Example - typeof Type Guard

```ts
function getId(value: string | number) {
  if (typeof value === 'string') {
    // value is string here
    console.log("ID (string):", value.toUpperCase());
  } else {
    // value is number here
    console.log("ID (number):", value.toFixed(0));
  }
}

getId('abc');   // ID (string): ABC
getId(123);     // ID (number): 123
```

TypeScript narrows the type based on the typeof check.

## Summary

- Type guard helps TypeScript know the exact type at runtime.
- It is used with typeof, instanceof, in, or custom functions.
- It makes your code safer and more reliable.

Type safely, Code smart!

> "Type guards turn uncertainty into confidence!"

# TYPESCRIPT TYPE GUARD

## Page 25 : Type Guard

Type guards help TypeScript understand the exact type at runtime!

## 1. instanceof Type Guard

- The instanceof operator is used to check whether an object is an instance of a specific class (constructor function).
- It helps TypeScript narrow down the type based on the class.
- It is commonly used with classes and built-in constructors (like Array, Date, etc.).

```ts
// Example: instanceof Type Guard
class Dog {
  bark() {
    return "Woof!";
  }
}
class Cat {
  meow() {
    return "Meow!";
  }
}

function makeSound(animal: Dog | Cat) {
  if (animal instanceof Dog) {
    console.log("Dog sounds:", animal.bark());
  } else if (animal instanceof Cat) {
    console.log("Cat sounds:", animal.meow());
  } else {
    console.log("Unknown animal");
  }
}

const pet: Dog | Cat = new Dog();
makeSound(pet);  // Dog sounds: Woof!
```

Here, TypeScript narrows the type based on the class using instanceof.

instanceof

### Quick Notes

- Works with classes and built-in constructors (Array, Date, etc.).
- Helps narrow down the type to a specific class.
- Useful when dealing with objects that have a common parent class or interface.

instanceof checks the class at runtime!

## 2. in operator Type Guard

- The in operator checks whether a property exists in an object.
- It helps TypeScript narrow down the type based on the presence of a property.
- It works well with union types where different types have different properties.

```ts
// Example: in operator Type Guard
interface User {
  name: string;
  age: number;
}
interface Admin {
  name: string;
  role: string;
}
type Person = User | Admin;

function printPerson(person: Person) {
  if ("role" in person) {
    // TypeScript knows it's Admin
    console.log("Admin role:", person.role);
  } else {
    // TypeScript knows it's User
    console.log("User age:", person.age);
  }
}

const user: Person = { name: "Ali", age: 25 };
printPerson(user);  // User age: 25
```

Here, TypeScript checks if "role" exists in the object and narrows the type.

in

### Important

- The in operator checks for property existence, not the value.
- It works with union types and objects.
- Useful when different types have different properties.

### Example - Practical Use

- Used in APIs, form data, or when working with optional properties in objects.

### Summary

- instanceof → checks class (constructor).
- in → checks property existence.
- Both help TypeScript narrow down the type at runtime and make your code safer.

> "Type guards = Smarter type checking + safer code!"

# TYPESCRIPT
# TYPE KEYOF OPERATOR

## Page 26 : keyof Operator

### Get Keys of a Type

The keyof operator lets you get the keys of a type as a union of string literals.

## 1. What is keyof Operator?

- The keyof operator is a type operator in TypeScript that creates a union of the keys of a given type.
- It helps you work with property names in a type-safe way.
- It returns a union of string (or number) literal types representing the keys of the type.

**Example (Basic Use)**

```ts
type Person = {
  name: string;
  age: number;
  email: string;
};

type PersonKeys = keyof Person;
// Result: "name" | "age" | "email"
```

keyof Person → "name" | "age" | "email"

## 1. keyof with Type Alias

- You can use keyof with a type alias to get the keys of the aliased type.
- It is useful when you want to create reusable types based on the keys.

```ts
// Type alias example
type User = {
  id: number;
  name: string;
  email: string;
};

type UserKeys = keyof User;
// Result: "id" | "name" | "email"

// Using the keys in a function
function getValue<T, K extends keyof T>(
  obj: T,
  key: K
): T[K] {
  return obj[key];
}

const user: User = {id: 1, name:"Ali", email:"a@x.com"};
const name = getValue(user, "name");
// name: string
```

Using keyof with a type alias gives you a union of keys.

## 2. keyof with Interface

- You can use keyof with an interface to get the keys of the interface.
- This is commonly used for safer property access and generic functions.

```ts
// Interface example
interface Product {
  id: number;
  title: string;
  price: number;
  inStock: boolean;
}

type ProductKeys = keyof Product;
// Result: "id" | "title" | "price" |
//         "inStock"

function getProductValue(
  product: Product,
  key: ProductKeys
) {
  return product[key];
}

const product: Product = {
  id: 1,
  title: "Laptop",
  price: 999,
  inStock: true
};

const title = getProductValue(product, "title");
// title: string
```

Works with interface too!

## 3. keyof with Classes

- You can also use keyof with a class to get the keys of the class properties.
- It includes only the public members of the class.

```ts
// Class example
class Car {
  brand: string;
  model: string;
  year: number;

  start(): void {
    console.log("Car started");
  }
}

type CarKeys = keyof Car;
// Result: "brand" | "model" | "year" |
//         "start"

function getCarValue(car: Car, key: CarKeys) {
  return car[key];
}

const myCar = new Car();
myCar.brand = "Toyota";

const brand = getCarValue(myCar, "brand");
// brand: string
```

Class keys include public members only.

## Quick Notes

- keyof returns a union of keys (property names).
- Works with type alias, interface, and classes.
- Only public members are included for classes.
- Very useful in generics, mapped types, and utility types.

## Summary

- keyof gets keys from a type, interface, or class.
- It returns a union of string (or number) literal types.
- Helps in creating reusable, type-safe code.
- Useful in generics, mapped types, and utility types.

Type safe code, better code!

> "Use keyof to turn your type into a list of keys!"

# TYPESCRIPT INDEX SIGNATURE

Page 27 : Index Signature

Dynamic Property Access

Index signature lets you define the type of properties that can be accessed using a string (or number) index.

## 1. What is Index Signature?

- An index signature allows you to specify the type of values for properties that are not explicitly declared in an object type.
- It is useful when the object will have unknown or dynamic keys (like a dictionary, map, or configuration object).
- You can use string or number as index signature (usually string is more common).

### Quick Notes

- Use [key: string] for string keys.
- Use [index: number] for numeric keys.
- All explicitly defined properties must match the index signature type (in most cases).

dynamic keys!

## 2. Example - Basic Index Signature

```
// Example: Basic index signature
interface StringDictionary {
  [key: string]: string;
}

const user: StringDictionary = {
  name: "Hussain",
  role: "Developer",
  department: "IT"
};

console.log(user["name"]);  // Hussain
console.log(user["role"]);  // Developer
```

Here, [key: string] means any string key will have a string value.

## 3. Index Signature with Number

```
// Example: Number index signature
interface NumberDictionary {
  [index: number]: string;
}

const colors: NumberDictionary = {
  0: "Red",
  1: "Green",
  2: "Blue"
};

console.log(colors[0]);  // Red
```

Here, [index: number] means any number key will have a string value.

123

## 4. Index Signature with Explicit Properties

```
// Explicit properties + index signature
interface UserInfo {
  name: string;
  age: number;
  [key: string]: string | number;
}

const user: UserInfo = {
  name: "Ali",
  age: 25,
  city: "Karachi",
  country: "Pakistan"
};

console.log(user.city);  // Karachi
```

All explicitly defined properties must be compatible with the index signature type (string | number in this case).

## 5. Important Points

- Index signature is used for dynamic property access.
- Use string or number as the index type.
- All explicit properties must match the index signature type (in most cases).
- It cannot be used with symbol (unless you use [key: symbol]).
- Be careful — it can make your type less strict if overused.

## 6. Practical Use

- Used for configuration objects (e.g., environment variables).
- Used in APIs when response objects have dynamic keys.
- Useful for dictionaries, maps, or data with unknown keys.

```
// Example: Environment variables
interface EnvConfig {
  [key: string]: string;
}

const env: EnvConfig = {
  API_URL: "https://api.example.com",
  PORT: "3000"
};
```

Useful for config objects or API data with dynamic keys.

## Summary

- Index signature allows dynamic property access.
- Use string or number as index type.
- All explicit properties must be compatible (usually).
- Great for dictionaries, configs, and dynamic data.

TypeScript = more flexibility + better type safety!

"Index signature gives flexibility to your objects with dynamic keys!"

# TYPESCRIPT UTILITY TYPES

Page 28

## Page 28 : Utility Types

### Build with Existing Types

Utility types help you create new types from existing types in a simpler and cleaner way.

### 1. What are Utility Types?

- Utility types are built-in helper types in TypeScript that let you transform, manipulate, and create new types from existing ones.
- They help you avoid repetition and make your code more concise and readable.
- They are part of TypeScript's advanced type system.

#### Quick Notes

- Utility types are predefined in TypeScript.
- They work with existing types (like interfaces, types, or classes).
- They are very useful in real-world projects for building flexible and reusable types.

Reuse & Simplify!

### 2. Partial<Type>

- Makes all properties of a type optional.
- Useful when you want to create a type for partial updates (like form data or config).

```ts
// Example: Partial
interface User {
  name: string;
  age: number;
  email: string;
}

// All properties become optional
const updateUser: Partial<User> = {
  name: "Hussain"  // only name is required
};
```

All properties are optional now!

### 3. Required<Type>

- Makes all properties of a type required.
- Useful when you want to ensure all properties are present.

```ts
// Example: Required
interface User {
  name?: string;
  age?: number;
  email?: string;
}

const completeUser: Required<User> = {
  name: "Hussain",
  age: 25,
  email: "hussain@example.com"
};
```

All properties are now required!

### 4. Readonly<Type>

- Makes all properties of a type readonly.
- Prevents modification of properties after creation.

```ts
// Example: Readonly
interface User {
  name: string;
  age: number;
}

const user: Readonly<User> = {
  name: "Hussain",
  age: 25
};

// This will give an error
user.name = "Ali";  // Error: Cannot assign
                    // to 'name' because it is readonly.
```

Properties cannot be changed!

### 5. Pick<Type, Keys>

- Creates a new type by selecting specific properties from an existing type.
- Useful for extracting only the needed fields.

```ts
// Example: Pick
interface User {
  name: string;
  age: number;
  email: string;
  role: string;
}

type UserNameAndEmail =
  Pick<User, 'name' | 'email'>;

const user: UserNameAndEmail = {
  name: "Hussain",
  email: "hussain@example.com"
};
```

Only selected properties are kept!

### 6. Omit<Type, Keys>

- Creates a new type by removing specific properties from an existing type.
- Useful when you want to exclude certain fields.

```ts
// Example: Omit
interface User {
  name: string;
  age: number;
  email: string;
  role: string;
}

type UserWithoutRole = Omit<User, 'role'>;

const user: UserWithoutRole = {
  name: "Hussain",
  age: 25,
  email: "hussain@example.com"
};
```

Removed 'role' property!

### 7. Exclude<Type, Excluded>

- Creates a new type by excluding specific types from a union.
- Useful for filtering out unwanted types.

```ts
// Example: Exclude
type Status = "pending" | "success" | "error";

type NotError = Exclude<Status, "error">;

const status: NotError = "success";
// status can be "pending" or "success"
// but not "error"
```

Removes 'error' from the union!

### 8. NonNullable<Type>

- Removes null and undefined from a type.
- Useful when you want to ensure a value is always present.

```ts
// Example: NonNullable
type MaybeString = string | null | undefined;

type SafeString = NonNullable<MaybeString>;

const name: SafeString = "Hussain";
// name can never be null or undefined
```

Removes null and undefined!

### Summary

- Utility types help you create new types from existing types.
- They make your code cleaner, more reusable and flexible.
- Common utility types: Partial, Required, Readonly, Pick, Omit, Exclude, NonNullable, etc.

> "Utility types turn existing types into exactly what you need!"

Smarter Types Better Code!

# TYPESCRIPT DECORATORS

## Page 29 : Decorators

### Add Extra Functionality

Decorators let you add extra functionality to classes, methods, properties, and more in a clean way.

### 1. What are Decorators?

- Decorators are a special type of function in TypeScript (ESNext feature) that can be attached to classes, methods, properties, or parameters.
- They are used to add metadata, modify behavior, or extend functionality.
- Decorators are a way to implement cross-cutting concerns like logging, validation, authentication, etc.

### Quick Notes

- Decorators start with the @ symbol.
- They run at runtime (after the class is defined).
- We need experimentalDecorators enabled in tsconfig.json.

@ symbol means Decorator!

### 2. Types of Decorators

- Class Decorator – Used with a class.
- Method Decorator – Used with a method.
- Property Decorator – Used with a property.
- Parameter Decorator – Used with a method parameter.

### Important

- Decorators can be used in combination
- and can be created as custom decorators for your own logic.

More control + More flexibility!

### Example – tsconfig.json

```
{
  "compilerOptions": {
    "target": "ESNext",
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

Enable experimental decorators!

### 3. Class Decorator

- Used to add metadata or modify the class itself.

```
// Class Decorator Example
function Logger(constructor: Function) {
    console.log("Class is being created");
}

@Logger
class User {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
}

const user = new User("Hussain");
```

Class Decorator

### 4. Method Decorator

- Used to modify or add behavior to a method.

```
// Method Decorator Example
function Log(target: any,
    propertyKey: string,
    descriptor: PropertyDescriptor) {
  const originalMethod =
    descriptor.value;
  descriptor.value = function (...args: any[]) {
    console.log(`Calling ${propertyKey}`);
    return originalMethod.apply(this, args);
  };
  return descriptor;
}
class Calculator {
  @Log
  add(a: number, b: number): number {
    return a + b;
  }
}
const calc = new Calculator();
calc.add(5, 3);
```

Method Decorator

### 5. Property Decorator

- Used to add metadata or modify a property.

```
// Property Decorator Example
function Required(target: any,
    propertyKey: string) {
  let value: any;
  const getter = function () {
    return value;
  };
  const setter = function (newVal: any) {
    if (newVal === undefined || newVal === null) {
      throw new Error(`${propertyKey}
        is required`);
    }
    value = newVal;
  };
  Object.defineProperty(target,
  propertyKey, { get: getter, set: setter });
}
class Product {
  @Required
  name: string;
}
const p = new Product();
p.name = "Laptop";
```

Property Decorator

### 6. Parameter Decorator

- Used to add metadata to a method parameter.

```
// Parameter Decorator Example
function ParamInfo(target: any,
    propertyKey: string,
    parameterIndex: number) {
  console.log(`Parameter index: ${parameterIndex}`);
}

class AuthService {
  login(@ParamInfo userId: string) {
    console.log(userId);
  }
}

const service = new AuthService();
service.login("user123");
```

Parameter Decorator

### Quick Notes

- Decorators are just functions.
- They can be used on classes, methods, properties, and parameters.
- They are widely used in frameworks like Angular, NestJS, etc.
- Always enable experimentalDecorators in tsconfig.json.

### Summary

- Decorators provide a powerful way to extend TypeScript classes and add extra functionality.
- They help in clean, reusable, and maintainable code.
- Useful for logging, validation, authentication, performance tracking, etc.

TypeScript + Decorators = Cleaner code + More power!

> "Decorators add power to your code!"

# TYPESCRIPT NON NULL ASSERTION OPERATOR

Non null assertion operator tells TypeScript that a value is not null or undefined!

## Page 30 : Non Null Assertion Operator

Use ! to tell TypeScript

### 1. What is Non Null Assertion Operator?

- The non null assertion operator (!) is used to tell TypeScript that a value is not null or undefined.
- It removes null and undefined from the type of a value.
- It is useful when you are sure about the value at runtime, but TypeScript cannot infer it.
- It is commonly used with optional properties, DOM elements, and function returns.

#### Quick Notes

- Uses the exclamation mark (!).
- Does not perform any runtime check.
- If the value is actually null or undefined, it can cause a runtime error.

Be sure about the value!

### 2. Basic Example

```
let user: { name: string } | null = { name: "Hussain" };

// Without non-null assertion (error)
console.log(user.name);
// Error: Object is possibly 'null'.

// With non-null assertion
console.log(user!.name);  // Works fine
```

! tells TypeScript this value is not null or undefined.

### 3. With Optional Chaining

- You can also use optional chaining (?.) when you want to safely access a property.
- But if you are sure the value is not null, you can use ! for direct access.

```
let user: { profile?: { name: string } } | null = {
  profile: { name: "Hussain" }
};

console.log(user?.profile?.name);  // Safe way
console.log(user!.profile!.name);  // Use ! when sure
```

?. is safe, ! is direct access.

### 4. In a Function Return

- The non null assertion operator is useful when a function can return null or undefined, but you are sure it will return a value.

```
function getUser(id: number): { name: string } | null {
  if (id === 1) {
    return { name: "Hussain" };
  }
  return null;
}

const user = getUser(1)!;
console.log(user.name);  // Works fine (assuming id = 1)
```

Tells TS return value is not null or undefined.

### 5. Real World Example (DOM Element)

- When working with DOM elements, TypeScript may consider them possibly null.
- Use ! when you are sure the element exists in the DOM.

```
const input = document.getElementById("name")!;
input.value = "Hussain";  // Works fine (if element exists)
```

DOM element may be null, use ! if you are sure.

### Summary

- The non null assertion operator (!) is used to remove null and undefined from a type.
- It helps when you are sure about the value at runtime.
- It does not perform any runtime check.
- Use it carefully, as it can cause runtime errors if the value is actually null or undefined.

! = Non null assertion operator (Use it when you are sure!)

> "Tells TypeScript that the value is not null or undefined."

# TYPESCRIPT BACKEND DEVELOPMENT

## Page 31 : tsconfig.json

Configure TypeScript for Backend

tsconfig.json helps TypeScript work better with Node.js and your project!

### 1. What is tsconfig.json?

- tsconfig.json is a configuration file for the TypeScript compiler (tsc).
- It tells TypeScript how to compile your code, where to find files, and which features to use.
- It helps you set up a consistent and maintainable project structure, especially in backend development with Node.js.

#### Quick Notes

- It is the main configuration file for TypeScript.
- It is optional, but highly recommended.
- When you run tsc, it looks for tsconfig.json in the project root.

Without it, TypeScript uses default settings!

### 2. Example: tsconfig.json File

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "es6",
    "target": "es6",
    // For nodejs:
    // "lib": ["esnext"],
    // "types": ["node"],
    // and npm install -D @types/node
    "strict": true
  }
}
```

This file controls how TypeScript compiles your code.

### 3. Explanation of Each Option

#### 1. rootDir: "./src"

- Specifies the root directory of your source files.
- All TypeScript files will be searched inside the ./src folder.

Your code lives here

./src

#### 2. outDir: "./dist"

- Specifies the output directory for compiled files.
- All JavaScript files will be placed inside the ./dist folder after compilation.

Compiled code will be here

./dist

#### 3. module: "es6"

- Defines the module system to use.
- "es6" means the output will use ES6 module syntax (import/export).
- This is modern and works well with bundlers and Node.js (with proper setup).

```
// output (ES6 module)
import { something } from "./file";
```

#### 4. target: "es6"

- Specifies the JavaScript version to compile to.
- ES6 is modern and supports features like let/const, arrow functions, classes, etc.

This does NOT mean your code will run only in ES6, it just tells TypeScript which version of JS to output.

#### 5. lib: ["esnext"] (For Node.js)

- Defines the library files to include in compilation.
- "esnext" includes the latest JavaScript features.
- Used when working with modern Node.js features.
- (Usually commented out in basic setups.)

Needs @types/node for Node.js types.

#### 6. types: ["node"] (For Node.js)

- Includes type definitions for Node.js (global, process, etc).
- Helps TypeScript understand Node.js APIs and gives better type checking.

Install with:
npm install -D @types/node

### 7. strict: true

- Enables all strict type checking options.
- Helps you catch errors early and write safer code.
- Recommended for production-level projects.

It enables (internally):

- noImplicitAny
- strictNullChecks
- strictFunctionTypes
- strictPropertyInitialization

and more...

More strict = More safe code!

### Summary

- tsconfig.json controls how TypeScript compiles your code.
- It helps you set up a clean and organized project structure.
- Use it to enable modern features and better type checking.
- It is especially important when working with Node.js and backend projects.

> A well-configured tsconfig.json makes your TypeScript backend project faster, cleaner and more reliable!

Better config → Better development → Better backend!

# TYPESCRIPT BACKEND DEVELOPMENT

## Page 32 : Script Setup

Understanding the npm Scripts in package.json

Scripts help us run useful commands for development, build and production in our TypeScript project

### 1. What is Script Setup?

- In a TypeScript backend project, we use the "scripts" section in package.json to define custom commands.
- These scripts make it easy to run the development server, compile TypeScript, and start the production build.
- Instead of typing long commands, we just run npm run <script-name>.

**Quick Notes**

- Scripts are defined inside the "scripts" object in package.json.
- They run using npm run <script-name>.
- They help in development, build and production workflow.

Short commands = Big productivity!

### 2. The Script Setup Code

```
"scripts": {
  "build": "tsc",
  "dev": "nodemon ./src/server.ts",
  "start": "node ./dist/server.js"
},
```

These are the scripts we will use in our project.

### 3. Explanation of Each Script

**1. "build": "tsc"**

Compile TypeScript code to JavaScript.

- Uses the TypeScript compiler (tsc).
- Reads the tsconfig.json file.
- Generates the output files in the dist folder (as configured).

Run Command: `> npm run build` (TS → JS)

**2. "dev": "nodemon ./src/server.ts"**

Run the development server with auto-restart.

- Uses nodemon to watch for file changes.
- Automatically restarts the app when you make changes.
- Works with the TypeScript file directly (./src/server.ts).

Run Command: `> npm run dev` — nodemon

**3. "start": "node ./dist/server.js"**

Run the built project in production.

- Executes the compiled JavaScript file from the dist folder.
- Used after running the build script.

Run Command: `> npm run start` — Run compiled code

### Example Project Workflow

1. npm run dev → Start development server (with auto-restart).
2. npm run build → Compile TypeScript to JavaScript (in dist folder).
3. npm run start → Run the production build.

### Why It's Useful?

- Saves time and avoids long commands.
- Keeps your workflow organized and simple.
- Supports development, build and production.
- Works great with tools like nodemon and tsc.
- Makes your project easy to manage and scale.

Build → Develop → Deploy

### Important Points

- Make sure tsconfig.json is correctly configured.
- Install nodemon as a dev dependency: npm install -D nodemon
- For using nodemon with TypeScript, you can also use ts-node:

```
npm run dev → nodemon --watch src --exec ts-node ./src/server.ts
```

> "With the right scripts, your TypeScript backend project becomes faster, simpler and more productive!"

Clean scripts = Happy development!

# TYPESCRIPT BACKEND DEVELOPMENT

## Page 33 : npm run build (Detailed Flow)

### Step by Step: How npm run build Works

- npm run build uses TypeScript compiler (tsc) to convert your TypeScript code into JavaScript code!

### 1. What is npm run build ?

- npm run build is a script command that runs the TypeScript compiler (tsc) to build your project.
- It reads the tsconfig.json file for configuration.
- It compiles your TypeScript (.ts) files into JavaScript (.js) files and places them in the specified output directory (e.g. dist).

**Quick Notes**

- The build command is usually defined in package.json.
- It helps in creating production-ready JavaScript code.
- It uses tsconfig.json settings like outDir, target, etc.

Build = Compile + Output

### 2. The Script in package.json

```
"scripts": {
  "build": "tsc",
  "dev": "nodemon ./src/server.ts",
  "start": "node ./dist/server.js"
},
```

- When you run npm run build, it executes tsc.
- tsc is the TypeScript compiler (installed via devDependencies).

### 3. What does tsc do?

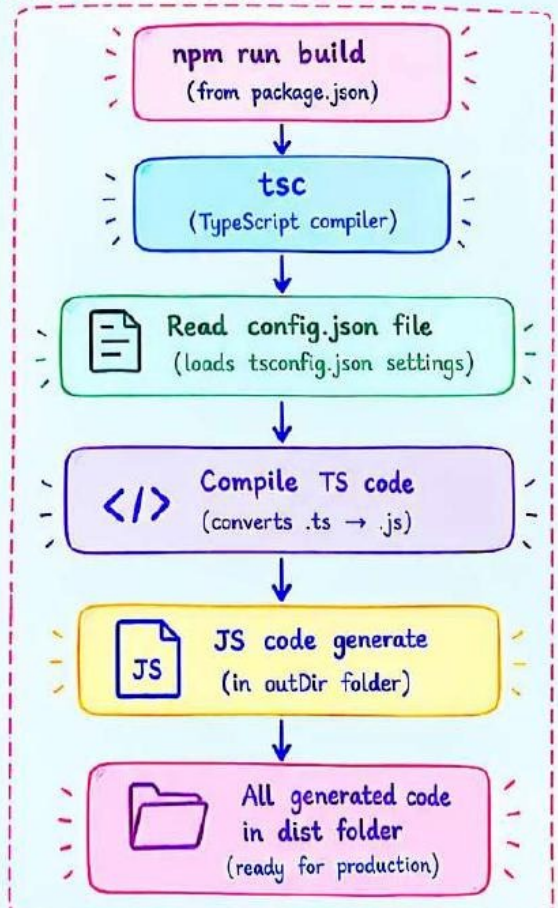
- Reads the tsconfig.json file.
- Uses the configuration (compilerOptions).
- Compiles all .ts files in your project.
- Generates .js files in the outDir folder (e.g. dist).
- Shows errors if there are any type issues.

tsc = TypeScript Compiler

### 4. Result

- After compilation, all your TypeScript files are converted into JavaScript files.
- These files are placed in the dist folder.
- Now you can run them with node (using npm run start).

### Flowchart : npm run build




### tsconfig.json (for reference)

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "es6",
    "target": "es6",
    // "lib": ["esnext"],
    // "types": ["node"],
    "strict": true
  }
}
```

### What happens in this example?

1. Root directory is src (your TypeScript source files).
2. Compiled files go to dist (output directory).
3. Module and target are set to ES6.
4. Strict mode is enabled for better type safety.
5. For Node.js, you can use lib: ["esnext"] and types: ["node"], and install @types/node as a dev dependency.

### Final Output Structure

```
project/
├ src/
│  └ server.ts
├ dist/
│  └ server.js
├ tsconfig.json
└ package.json
```

"Build your TypeScript code, then run it with Node.js!"

# TYPESCRIPT BACKEND DEVELOPMENT

## Page 34 : npm run dev (Detailed Flow)

### Step by Step: How npm run dev Works

npm run dev
uses nodemon + ts-node
to run your TypeScript
code in development
mode.

### 1. What is npm run dev?

- npm run dev is a script command that runs your TypeScript project in development mode.
- It uses nodemon to watch for file changes and runs your server with ts-node (which understands TypeScript).
- This gives you auto-restart on changes, so you don't have to stop and start the server manually.

### Quick Notes

- nodemon watches your files for changes.
- ts-node runs TypeScript directly (no manual build).
- tsconfig.json is used to configure the TypeScript compiler.
- It's perfect for development (not for production).

Auto Restart on Changes!

### The Script in package.json

```
"scripts": {
 "dev": "nodemon ./src/server.ts"
}
```

- This tells npm to run nodemon with your TypeScript entry file.
- nodemon will restart the server whenever the file changes.

### What is nodemon?

- nodemon is a development tool.
- It watches for changes in your files.
- When it detects a change, it restarts the application.
- Here it watches the .ts file (server.ts).

nodemon

Watch & Restart!

### What is ts-node?

- ts-node runs TypeScript files directly (without pre-compiling).
- It reads tsconfig.json to know how to compile.
- It is used here by nodemon to execute your .ts file.

ts-node

Run .ts directly!

### Why This is Useful?

- You get instant feedback while developing.
- No need to build manually.
- Simpler and faster development experience.
- Automatically restarts on file changes.

Develop Faster!

### The Flowchart

1. **npm run dev**
   You run this command in your terminal.
2. **nodemon ./src/server.ts**
   npm runs nodemon with your entry file.
3. **nodemon detects .ts file**
   It watches for changes in ./src/server.ts.
4. **call ts-node**
   nodemon calls ts-node to run the .ts file.
5. **ts-node read tsconfig.json file**
   ts-node reads tsconfig.json to get compiler options (like module, target, etc).
6. **compile TS code**
   TypeScript code is compiled according to the settings in tsconfig.json.
7. **JS code generated**
   Compiled JavaScript code is created (in memory).
8. **give this generated JS code to nodejs**
   ts-node passes the compiled JS code to Node.js.
9. **nodejs run this code**
   Node.js executes the JavaScript code and your server starts.

- npm run dev
  = nodemon + ts-node
  = Auto Restart + Faster Development!

> "TypeScript + nodemon + ts-node
> = Smooth Development Experience!"

# TYPESCRIPT BACKEND DEVELOPMENT

## Page 35 : npm run start (Detailed Flow)

Step by Step: How npm run start Works

npm run start runs your built JavaScript file using Node.js (in production)

### 1. What is npm run start?

- npm run start is a command that runs your compiled (built) JavaScript code using Node.js.
- It is used after the build step (npm run build).
- It helps you start your backend server in production mode (or normal mode).

**Quick Notes**

- It runs the file specified in package.json (start script).
- Usually points to the compiled JS file in dist folder.
- Does NOT compile TypeScript (it uses already compiled JS).

Run Your App!

### 2. The Script in package.json

```
"scripts": {
  "build": "tsc",
  "dev": "nodemon ./src/server.ts",
  "start": "node ./dist/server.js"
}
```

- npm run start runs the node command with the file ./dist/server.js.
- This file is the compiled JavaScript version of your TypeScript server code.

### 3. What happens in this example?

1. npm run start → runs the start script (node ./dist/server.js).
2. Node.js → loads and executes the JS file.
3. Your backend server starts and listens for requests.

Your server is now running!

### Full Example in Practice

1. Build (once):
   > npm run build
   (generates JS in dist/)

→

2. Start the server:
   > npm run start
   (runs dist/server.js with Node.js)

Build → Start → Your TypeScript backend is ready!

### 4. Flowchart : npm run start

1. npm run start (from package.json)
2. node ./dist/server.js — Node.js runs this file.
3. Loads the compiled JS file (from dist folder)
4. Your backend server starts (e.g. Express server)
5. Server listens for requests (e.g. http://localhost:3000)

**Important Points**

- npm run start does NOT compile TypeScript.
- It only runs the already compiled JS file from the dist folder.
- Make sure you run npm run build first (to generate JS).
- In production, you usually use npm run start to run the final build.

> "npm run start makes your TypeScript backend application live and ready to use!"

Simple steps → Powerful backend!

# TYPESCRIPT BACKEND DEVELOPMENT

## Page 36 : nodemon & ts-node

Tools that make TypeScript backend development easier!

nodemon & ts-node work together to give you a better development experience in TypeScript backend projects!

## 1. nodemon

- nodemon is a development tool that automatically restarts your server when it detects file changes.
- It watches your files (e.g. .ts files) and re-runs your application without you having to manually restart it.
- It is very useful in development mode (not needed in production).

### Quick Notes

- Watches file changes in your project.
- Automatically restarts the server.
- Works with Node.js and TypeScript (with ts-node).
- Helps you save time during development.

Auto Restart on Changes!

### How nodemon works (Internally)

1. **Start**
   You run:
   `nodemon ./src/server.ts`
2. **Watch files**
   nodemon watches your project files (using file system events).
3. **File change detected**
   When a file changes (e.g. .ts file), nodemon detects it.
4. **Restart process**
   It kills the current process and restarts the command (ts-node).

**In short:**
nodemon = Watches files + Restarts your server

## 2. ts-node

- ts-node is a Node.js execution environment for TypeScript.
- It allows you to run TypeScript files directly (without compiling them manually).
- It works by hooking into Node.js and using the TypeScript compiler (tsc) internally.

### Quick Notes

- Runs .ts files directly.
- Uses the TypeScript compiler (tsc) under the hood.
- Reads tsconfig.json for configuration.
- Very useful for development and testing.

Run TS Directly!

### How ts-node works (Internally)

1. **You run**
   `nodemon ./src/server.ts`
   (or directly `ts-node ./src/server.ts`)
2. **ts-node reads tsconfig.json**
   It loads your TypeScript configuration (e.g. module, target, strict, etc).
   tsconfig.json
3. **Compile TypeScript**
   ts-node uses the TypeScript compiler (tsc) to compile your .ts code in memory (not to disk).
4. **Execute with Node.js**
   After compilation, it gives the generated JavaScript code to Node.js to run.

**In short:**
ts-node = Compiles TS in memory + Runs with Node.js

### Key Differences (Quick View)

| Feature | nodemon | ts-node |
|---|---|---|
| Purpose | Auto-restart on file changes | Run TS directly |
| What it does | Watches files & restarts process | Compiles & executes TS |
| Works with | JS / TS (any file) | Only TS (with Node.js) |
| Uses tsconfig.json | No | Yes |
| Needed in | Development | Development (and testing) |

### Together

In development, we use both:

1. nodemon (watches files & restarts)
2. ts-node (compiles & runs TS)

So, your server automatically restarts and runs your latest TypeScript code whenever you make changes!

**Result:**
Fast + Easy + Productive!

Develop faster with TypeScript + Node.js !

# TYPE DEFINITION PACKAGE
# @types/express

Page 37 : Type Definition Package

Provides TypeScript type information for npm packages

Type definition packages provide only type info (.d.ts files) — no runtime code!

## 1. What is a Type Definition Package?

- A type definition package contains TypeScript declaration files (.d.ts) that provide type information for a JavaScript library.
- It does NOT include the actual runtime code.
- @types/express; gives TypeScript types for the Express npm package.
- This helps you get proper type checking and IntelliSense in your editor.

### Quick Notes

- Type definition packages are usually named as @types/<package>.
- They are installed as devDependencies.
- They only provide types, not actual functionality.

Types only! No runtime!

## 2. Installation

To get TypeScript types for Express, run:

```
npm i --save-dev @types/express
```

--save-dev means it's only for development (used in dev and testing).

### How it works (Flow):

1. Install express (runtime package).
2. Install @types/express (type definitions).
3. TypeScript finds declarations in node_modules/@types.
4. Editor/compiler gets types + IntelliSense.
5. Runtime still uses the real express package.

## 3. Code Example

```
import express, { Express, Application } from "express";

// const app: Express = express();
const app: Application = express();
```

### What's happening?

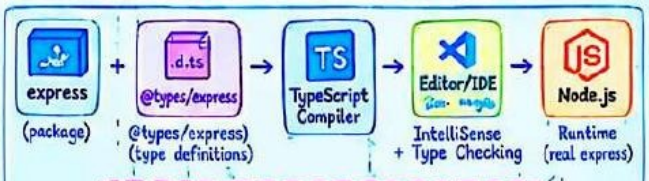
- express → default import (function) used to create the app.
- { Express, Application } → TypeScript types from @types/express.
- Application → specific type for the Express app (recommended).
- Express → another type representing the Express application.
- The type annotation is for compile-time checking, not runtime.

## 4. Application Type Comparison

| Application | Express |
|---|---|
| Represents the Express application instance. | Also represents the Express application instance. |
| Both are TypeScript types (from @types/express). | Both are TypeScript types (from @types/express). |
| Used for type annotation (e.g. const app: Application). | Used for type annotation (e.g. const app: Express). |
| Exact exported type surface comes from the installed Express declarations. | Exact exported type surface comes from the installed Express declarations. |

In short: Application and Express are just TypeScript type names for the same thing.

## 5. Internal Working (Under the Hood)




Types help at development time, not at runtime!

## 6. Practical Backend Example

```
import express, { Application } from "express";

const app: Application = express();

app.get("/", (req, res) => {
  res.send("Hello");
});
```

Simple Express server with types!

Runtime (real express)

Server

## 7. Summary

- @types/express provides TypeScript types for Express.
- Install with: npm i --save-dev @types/express
- Use Application or Express for type annotation.
- It helps with type checking, IntelliSense and better DX.
- It does not include runtime code.

> Type definitions make your Express backend safer, smarter and more productive!

# TYPESCRIPT BACKEND
# .ENV FILE & DOTENV

Page 38

.env + dotenv = easy & secure configuration in TypeScript backend!

## 1. What is .env?

- A .env file stores environment variables as KEY=VALUE pairs, commonly used for configuration such as PORT, database URL, API keys, etc.

Example:

```
PORT=5000
DB_URL=mongodb://localhost:27017/mydb
NODE_ENV=development
```

**Note:** keep secrets out of source code and normally add .env to .gitignore. Values are read as strings.

## 2. Install dotenv

Command:

```
npm install dotenv
```

- Dotenv loads variables from a .env file into process.env.
- It is a runtime package, so install it as a normal dependency when your app uses it.

## 3. Code Example

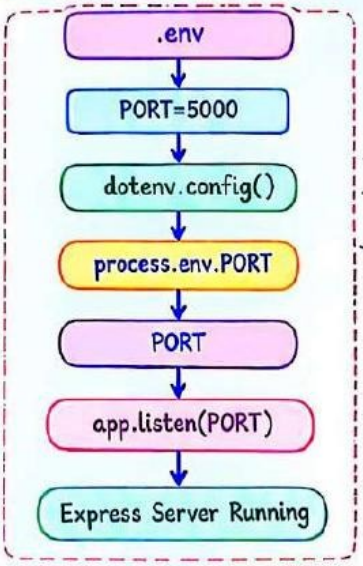
```
import express, { Express, Application } from "express";
import dotenv from "dotenv";

dotenv.config();

// const app: Express = express();
const app: Application = express();

const PORT = process.env.PORT!;

app.listen(PORT, () => {
  console.log(`server is running on http://localhost:${PORT}`);
});
```

TypeScript + Express

## 4. Code Explanation

1. import dotenv from "dotenv" → imports dotenv.
2. dotenv.config() → reads .env and puts its values into process.env.
3. process.env.PORT → gets the PORT environment variable.
4. PORT! → TypeScript non-null assertion; tells TypeScript to treat PORT as present. It does NOT create or validate the variable at runtime.
5. app.listen(PORT, ...) → starts the Express server on that port.
6. console.log(...) → prints the server URL.

## 5. Flow




## 6. Quick Notes

- dotenv.config() should run before code that relies on these environment variables.
- .env is configuration, not TypeScript code.
- process.env values are strings.
- For production, environment variables are often supplied by the hosting environment instead of a local .env file.

Keep Secrets Safe!

## 7. Summary

| | |
|---|---|
| .env | = stores configuration |
| dotenv | = loads .env |
| process.env | = accesses environment variables |
| app.listen() | = starts server |

Simple Config Big Impact!

# MONGODB CONNECTIVITY (TypeScript)

## Page 39 : MongoDB Connection & dbConnect()

Connect your TypeScript app to MongoDB Database!

### What is this code?

- This code establishes a connection with MongoDB using Mongoose.
- It sets custom DNS servers, reads the MongoDB URL from .env file,
- handles errors properly and logs the connection status.

### Prerequisite

- Install Mongoose: `npm i mongoose`
- Set your MongoDB URL in .env file (`MONGODB_URL=mongodb+srv://<username>:<password>@...`)

### Complete Code (Single Unit)

```
1  import mongoose from "mongoose";
2  import dns from "dns";
3
4  // Set custom DNS servers (Google & Cloudflare)
5  dns.setServers(["8.8.8.8", "1.1.1.1"]);
6
7  export const dbConnect = async () => {
8    try {
9      const MONGODB_URL = process.env.MONGODB_URL;
10     // Check if URL exists
11     if (!MONGODB_URL) {
12       process.exit(1);
13     }
14
15     // Connect to MongoDB
16     const connectionInstance = await mongoose.connect(MONGODB_URL);
17
18     // Log database name from connection
19     console.log(connectionInstance.connection.name, "Database Connected");
20
22   } catch (error) {
23     // Handle errors properly
24     if (error instanceof Error) {
25       console.error("MongoDB Connection Error:", error.message);
26       process.exit(1);
27     } else {
28       console.error("MongoDB Connection Unknown Error:", error);
29       process.exit(1);
30     }
25   }
};
```

### Step-by-Step Explanation

**1 Import modules**
- mongoose → MongoDB ODM (for Node.js).
- dns → Node.js built-in module to set custom DNS servers.

**2 Set custom DNS servers**
- `dns.setServers(["8.8.8.8", "1.1.1.1"]);`
- Uses Google & Cloudflare DNS to resolve domain names (e.g. MongoDB cluster).

**3 Get MongoDB URL**
- `const MONGODB_URL = process.env.MONGODB_URL;`
- Read the connection string from .env file.

**4 Check if URL exists**
- If not, exit the process (`process.exit(1)`).

**5 Connect to MongoDB**
- `await mongoose.connect(MONGODB_URL);`
- Establishes the connection using Mongoose.

**6 Log success**
- `console.log(connectionInstance.connection.name, "Database Connected");`
- Shows the name of the connected database.

**7 Handle errors**
- If error is an instance of Error → log message.
- Else → log unknown error.
- Then exit the process.

### Key Points

- Use Mongoose for MongoDB connection.
- Custom DNS servers help in domain resolution.
- MONGODB_URL is stored in .env file (not in code).
- Always handle errors properly.
- Exit the process if connection fails.

### .env File (Example)

```
MONGODB_URL=mongodb+srv://<username>:<password>@cluster0.mongodb.net/mydb
```

**Note:**
- Keep sensitive data in .env file.
- Do not hardcode credentials.
- Add .env to .gitignore.

### In Short

This code reads the MongoDB connection string from .env, sets DNS servers, connects to MongoDB using Mongoose, logs the database name and handles errors properly.

MongoDB Connected!

# MONGODB MODEL (Mongoose + TypeScript)

Page 40 : MongoDB Model

MongoDB.

Model is the structure of your data (collection schema) in MongoDB.

## What is a Model?

- A Model in Mongoose is a wrapper around a MongoDB collection.
- It defines the structure (schema) of the documents and provides an interface to interact with the database.
- In TypeScript, we use interfaces to define the shape of the document for type safety.

Schema → Model → Collection (in MongoDB)

- Schema: Defines structure
- Model: Provides methods (e.g. create, find, etc.)
- Collection (in MongoDB): Stores data

## The Code (Single Unit)

```ts
import mongoose, { Document } from "mongoose";

interface TodoSchema extends Document {
  todoName: string;
}

const todoSchema = new mongoose.Schema<TodoSchema>(
  {
    todoName: {
      type: String,
      trim: true,
      required: true,
    },
  },

  { timestamps: true },
);

export const TodoModel = mongoose.model<TodoSchema>("Todo", todoSchema);
```

### 1 Importing Modules

- mongoose → provides MongoDB functionality.
- { Document } → used for typing the document in TypeScript.

### 2 Interface (TypeScript)

- extends Document → adds built-in properties and methods (like _id, etc.) from Mongoose.
- todoName: string → defines the field in the document.

### 3 Creating Schema

- new mongoose.Schema<TodoSchema>(...) → defines the structure of the collection.
- todoName → field name.
- type: String → data type.
- trim: true → removes extra spaces.
- required: true → makes the field mandatory.
- { timestamps: true } → automatically adds createdAt and updatedAt fields.

### 4 Creating Model

- mongoose.model<TodoSchema>("Todo", todoSchema) → creates a model named "Todo" based on the schema.
- This model is used to perform CRUD operations on the collection.

## Line by Line Explanation

1. Import mongoose and Document from mongoose.
2. Define interface TodoSchema with a todoName field of type string.
3. Create a new schema using TodoSchema interface.
4. Define todoName field with type, trim and required options.
5. Enable timestamps (createdAt and updatedAt).
6. Create and export the TodoModel with the name "Todo".

## Result

- You get a model (TodoModel) which you can use to:

```ts
TodoModel.create({ todoName: "Learn TS" })
TodoModel.find()
TodoModel.findById("123")
TodoModel.updateOne(...)
and more...
```

## Key Points

- Document → gives TypeScript type safety.
- Schema → defines structure and validation rules.
- Model → provides methods to interact with DB.
- timestamps → automatically manages createdAt & updatedAt.
- Use interfaces for better type checking in TypeScript.

## In Short

- This code creates a MongoDB model called "Todo" with a single field "todoName" (string), adds timestamps, and uses TypeScript for type safety.

## Quick Note

- You can add more fields in the schema and define more validation rules, indexes, virtuals, etc. according to your needs.

Model = Schema + Collection + Type Safety + Easy Data Access ✓

Build Better Apps!

# CONTROLLER (BUSINESS LOGICS)

## (TypeScript Backend)

Page 42 : Controller Function (getTodos)

Controller handles the business logic and connects the models with routes.

### What is a Controller?

- A controller contains the business logic of your application.
- It receives the request from the route, interacts with the model (database) and sends the response back to the client.
- It keeps your routes clean and separates logic from routing.

### In this code

- getTodos is a controller function.
- It fetches all todos from MongoDB using TodoModel.
- It sends a JSON response to the client.
- It handles errors with try...catch.

### Controller Function Code (getTodos)

```
1  import type { Request, Response } from "express";
2  import { TodoModel } from "../models/todo_model.ts";
3
4  export const getTodos = async (req: Request, res: Response):
5    Promise<void> => {
6      try {
7        const allTodos = await TodoModel.find({});
8
9        res.status(200).json({
10         success: true,
11         message: "Successfully fetched all tasks!",
12         data: allTodos,
13       });
14     } catch (error) {
15       console.error(error);
16       res.status(500).json({
17         success: false,
18         message: "Internal Server Error!",
19       });
20     }
21 };
25
```

### Code Explanation

**1 Import Statements**

- import type { Request, Response } from "express";
  - Imports TypeScript types for Request and Response from Express.
- import { TodoModel } from "../models/todo_model.ts";
  - Imports the TodoModel (MongoDB model) from your model file.

**2 Controller Function Declaration**

- export const getTodos = async (req: Request, res: Response): Promise<void> => {
  - Defines an async function getTodos.
  - Takes req and res objects from Express.
  - Returns a Promise<void> (no value is returned).

**3 Try Block (Main Logic)**

- const allTodos = await TodoModel.find({});
  - Fetches all documents from the todos collection in MongoDB.
  - Returns an array of todos.

**4 Success Response**

- res.status(200).json({ ... });
  - Sends a 200 OK response with success message and the fetched data.

**5 Catch Block (Error Handling)**

- console.error(error);
  - Logs the error in the console.
- res.status(500).json({ ... });
  - Sends a 500 Internal Server Error response with an error message.

### How It Works (Flow)

Client Request (GET /todos) → Route (Calls getTodos) → Controller (getTodos) → TodoModel (find({})) → Response (JSON)

### Key Points

- Uses TypeScript types (Request, Response).
- Uses async/await for database operation.
- Uses MongoDB model (TodoModel).
- Sends proper HTTP status codes (200, 500).
- Handles errors with try...catch.

### Important

- Always validate and sanitize input (from client).
- Use proper error handling.
- Keep controllers focused on business logic only.
- Don't put model or route logic inside controller.

### Example Response

```
{
  "success": true,
  "message": "Successfully
             fetched all tasks!",
  "data": [
    { "_id": "...", "todoName": "Learn TS" },
    { "_id": "...", "todoName": "Build API" }
  ]
}
```

(Array of todos)

**In short:** The controller receives the request, fetches data from MongoDB using the model, then sends a JSON response to the client, while handling any errors. ✓

Keep Building

# Type Import / Export (TypeScript)

Page 43 : Type Import / Export

Import & Export help us share types, interfaces, functions and other values between files in TypeScript.

## What is Import / Export?

- `export` is used to make a variable, function, class, interface, type or value available to other files (modules).
- `import` is used to bring that exported item into another file.
- It helps in code organization and reusability.
- In TypeScript, we can import/export types, interfaces, classes, functions, etc.

## Types of Exports

1. Named Export – `export { ... };`
2. Default Export – `export default ...;`
3. Re-Export – `export * from 'module';`
4. Type Export – `export type { ... };`
5. Interface Export – `export interface { ... };`

## Example 1: Exporting an Interface (Type)

**File: apiSuccessResponseType.ts**

```ts
export interface ApiResponse<T> {
  success: boolean;
  message: string;
  data?: T;
}
```

**Explanation:**

- This file exports an interface named ApiResponse.
- T is a generic type, so the data field can be of any type.
- data? means it is optional.

## Example 2: Importing the Interface (Type)

**File: controller.ts**

```ts
import type { ApiResponse } from
  "../types/apiSuccessResponseType.ts";

// Now we can use the type like this
const example: ApiResponse<string> = {
  success: true,
  message: "Data fetched successfully!",
  data: "Some data"
};
```

**Explanation:**

- `import type` is used because we are importing a type, not a value (it's removed during compilation).
- The path points to the file where the interface is exported.

## Complete File Example

**1. apiSuccessResponseType.ts (Export)**

```ts
export interface ApiResponse<T> {
  success: boolean;
  message: string;
  data?: T;
}
```

**2. controller.ts (Import & Use)**

```ts
import type { ApiResponse } from
  "../types/apiSuccessResponseType.ts";

const res: ApiResponse<string> = {
  success: true,
  message: "Operation successful!",
  data: "Hello TypeScript"
};
```

## Other Import / Export Examples

**1. Exporting a function**

```ts
export const greet = (name: string) => {
  return `Hello, ${name}`;
};
```

**2. Importing a function**

```ts
import { greet } from "../utils/helper.ts";
console.log(greet("Hussain"));
```

**3. Default export**

```ts
export default class User { ... }
import User from "../user.ts";
```

## Key Points

- Use `export` to make things available to other files.
- Use `import` to use those exports in another file.
- Use `import type` for types/interfaces (removed at compile time).
- Use `type` or `interface` (both work with type-only imports).
- Organize your code with separate files for better maintainability.

## Why Use It?

- Keeps your code modular.
- Avoids duplicate code.
- Helps with type safety and better development experience.
- Makes your project easy to maintain and scale.

## Quick Note

When using import type, the imported type is not included in the compiled JavaScript. It only exists during type checking.

**In short :** export makes things available, import brings them in. Use import type for types, and keep your code modular and clean. ✓

TypeScript = Clean Code + Better Projects

# React Project Setup with TypeScript

Build modern UI with React and type safety with TypeScript!

## Page 01 : React + TypeScript Project Setup

### What is React?

- React is a popular JavaScript library developed by Facebook for building user interfaces.
- It uses a component-based architecture, which makes your code reusable and maintainable.
- React works with virtual DOM for better performance.

### What is TypeScript?

- TypeScript is a superset of JavaScript.
- It adds static typing to JavaScript.
- It helps catch errors early and improves code quality, maintainability and developer experience.
- It works perfectly with React.

### Why use TypeScript with React?

- Type safety (less runtime errors)
- Better IDE support (autocomplete, intellisense)
- Easier to maintain large codebases
- Clearer code and improved developer experience

### Project Setup Steps

1. Create a new React project using Vite (with TypeScript template).
2. Navigate to the project folder.
3. Install dependencies.
4. Run the development server.
5. Open the project in your editor (VS Code).

### Prerequisites

- ✓ Node.js (v18+ recommended)
- ✓ npm (comes with Node.js)
- ✓ VS Code (recommended)

### Note:

Vite is used here because it is fast, modern and supports TypeScript out of the box.

### 1 Create a project (Vite + TypeScript)

```
npm create vite@latest my-react-app -- --template react-ts
# OR using yarn
yarn create vite my-react-app --template react-ts
```

### 2 Navigate to project folder

```
cd my-react-app
```

### 3 Install dependencies

```
npm install
# or
yarn
```

### 4 Run development server

```
npm run dev
# or
yarn dev
```

### 5 Open in VS Code

```
code .
```

### Project Structure

```
my-react-app/
├── node_modules (dependencies)
├── public/ (static files)
├── src/
│   ├── assets/ (images, icons, etc.)
│   └── components/ (reusable components)
│   ├── App.tsx (main component)
│   └── main.tsx (entry file)
├── .gitignore
├── index.html
├── package.json
├── tsconfig.json (TypeScript config)
└── vite.config.ts (Vite config)
```

### Key Files

- tsconfig.json → TypeScript configuration.
- vite.config.ts → Vite build configuration.
- main.tsx → React entry point.
- App.tsx → Root component.

### Result ✓

You now have a React project with TypeScript set up and ready to code!

### Important Points

- Use the React + TypeScript template while creating the project.
- Do not manually install React or TypeScript separately (Vite handles it).
- Always use npm run dev to start the project.
- Keep your code inside the src folder.

### Example: Hello World (src/App.tsx)

```tsx
import { useState } from 'react';

function App(): JSX.Element {
  const [message, setMessage] = useState<string>('Hello TS');
  return (
    <div>
      <h1>{message}</h1>
    </div>
  );
}

export default App;
```

Happy Coding!

# React Props Data (with Proper Type)

TypeScript helps us define prop types, making our components safer and more reliable!

## Page 08 : Receiving Props Data with Proper Type

### What is it?

- In React, we pass data from a parent component to a child component using **props**.
- With TypeScript, we define the type of props using an interface.
- This helps in type safety, better code completion and fewer runtime errors.

### Key Points

- Props are read-only (cannot be changed in child).
- We use an interface to define the props type.
- Destructuring is used to access props in the component.
- TypeScript ensures the correct data type is passed.

### Syntax Overview

```
const Component: React.FC<{ prop1: type; ... }>
    = ({ prop1, .... }) => { ... };
```

### Prerequisites

- ✔ React + TypeScript project setup
- ✔ Basic knowledge of components & props
- ✔ TypeScript installed and configured

### Code Example

```tsx
import React from "react";

interface CardProps {
  readonly name: string;
  readonly age: number;
}

export default function Card({ name, age }: CardProps) {
  return (
    <div>
      <h1>name:- {name}</h1>
      <p>His age is:- {age}</p>
    </div>
  );
}
```

### Explanation

1. **Interface for Props**
   CardProps defines the type of props that this component will receive.
2. **Readonly Props**
   name and age are marked as readonly, so they cannot be changed inside the component.
3. **Destructuring**
   We access the props using destructuring and show the name and age in the UI.

### Props Data (Example)

```
<Card name="Hussain" age={20} />
```

→ This will pass name and age to the Card component with correct types.

### Benefits

- Type safety
- Better code completion
- Easier debugging
- Clean and maintainable code

### Quick Note

- Always define prop types using interfaces or type aliases in TypeScript for better development experience.

### In Short

We receive data through props, define their types using an interface, and use them safely in the child component.

**In short:** Props + TypeScript = Type Safe, Clean & Reliable Components. ✓

# Alternative Props Type Example

TypeScript helps us write cleaner, safer and more maintainable React code!

Page 09 : Alternative Way to Type Props

## What is it?

- This is another way to type props in React using React.FC (Functional Component).
- It's a generic type provided by React for function components.
- It helps in type safety, better code completion and fewer runtime errors.

## Key Points

- React.FC<{ ... }> is a generic type.
- The object inside defines the props type.
- It makes your component type-safe.
- You can also use defaultProps with it (if needed).

## Syntax

```
const Component: React.FC<{ prop1: type; ... }>
    = ({ prop1, ... }) => { ... };
```

## Prerequisites

- ✓ React + TypeScript setup
- ✓ Basic knowledge of functional components and props.

## Code Example

```
1  import React from "react";
2
3  const Card: React.FC<{ name: string; age: number }> =
4    ({ name, age }) => {
5      return (
6        <div>
7          <h1>name:- {name}</h1>
8          <p>His age is:- {age}</p>
9        </div>
10     );
11 };
12
13 export default Card;
14
```

## Explanation

1. `React.FC<{ name: string; age: number }>`
   → Defines the props type using a generic type.
2. `{ name, age }`
   → Destructures the props inside the component.
3. `export default Card`
   → Exports the component as default.

## Props Type (Example)

`{ name: string; age: number }`

→ This will pass name and age to the Card component with correct types.

## Benefits

- Type safety
- Better code completion
- Easier debugging
- Clean and maintainable code

## Quick Note

- You can also use React.FC<Props> or type Props = { ... } inside the component – both are valid, choose what you prefer.

## In Short

React.FC<{ ... }> is a simple and effective way to type props in React with TypeScript.

**In short:** React.FC<{ ... }> = Typed Props + Simple + Clean + Type Safe! ✓

# State Type and EventObj Type Management (React + TypeScript)

TypeScript helps us write safer React code with proper types!

## Page 10 : Managing State Type & Event Object Type

### What is it?

- In React with TypeScript, we use proper types for state and event objects to get type safety, better autocomplete and avoid runtime errors.
- useState<T>() helps us define the type of state.
- Event handlers use specific event types like React.ChangeEvent<HTMLInputElement>.

### Key Points

- State type ensures the state holds the correct data type.
- Event object type provides proper type for the event passed in the handler.
- Helps in better code completion and prevents type-related errors.

### Syntax

```
const [state, setState] = useState<Type>(initialValue);
function handleEvent(e: React.ChangeEvent<ElementType>): void
```

← Generic type for state

← Specific event type

### Prerequisites

- ✓ React + TypeScript setup
- ✓ Basic knowledge of hooks (useState)
- ✓ Understanding of event handling in React.

### Code Example

```tsx
import React, { useState } from "react";
import { ChangeEvent } from "react";

export default function TaskInput() {
  const [task, setTask] = useState<string>("");

  function handleInputChange(e: ChangeEvent<HTMLInputElement>): void {
    setTask(e.target.value);
  }

  return (
    <div>
      <input type="text" value={task} onChange={handleInputChange} />
      <p>Task: {task}</p>
    </div>
  );
}
```

### Explanation

1. **State Type**
   - useState<string>("")
   - The state "task" will always be a string.
2. **Event Object Type**
   - React.ChangeEvent<HTMLInputElement>
   - Gives proper type to the event object (e).
3. **Handler Function**
   - handleInputChange receives the typed event and updates the state with e.target.value.

### EventObj Type Example

React.ChangeEvent<HTMLInputElement>
→ Used for input, textarea, etc.

### Benefits

- Type safety for state and events
- Better autocomplete & IDE support
- Fewer runtime errors
- Clean and maintainable code

### Quick Note

- Always use the correct TypeScript types for state and event objects to make your React code more safe and reliable.

### In Short

Use proper types for state and event objects in React with TypeScript to get safer, cleaner and error-free code.

**In short:** State<Type> + React.ChangeEvent<ElementType> = Type Safe React Code! ✓

# Function Component Props (with children)

TypeScript helps us write safer React code with proper types!

## Page 11 : Passing Functions as Props in React

### What is it?

- A function component is just a JavaScript function that returns JSX.
- We can pass data to it using props.
- Props can be normal values, functions, or even children (React elements).
- The children prop is a special prop that represents the content between the opening and closing tags.

### Key Points

- Function components receive props as a single object.
- We define the type of props using TypeScript (interface or React.FC).
- children is typed as React.ReactNode (because it can be anything renderable).
- It allows us to create reusable and flexible components.

### Syntax

```
function ComponentName({ prop1, prop2 }: PropsType) {
  return (
    <div>
      {/* use props here */}
    </div>
  );
}
```

Function receives props as an object

### Props Type Example

```
interface CardProps {
  children: React.ReactNode;
}

function Card({ children }: CardProps) {
  return <div className="card">{children}</div>;
}
```

### children Prop Example

```
1  import React from "react";
2  interface CardProps {
3    children: React.ReactNode;
4  }
6  function Card({ children }: CardProps) {
7    return (
8      <div className="card" style={{ padding: "16px", border: "1px solid #ccc" }}>
9        <h2>Card Component</h2>
10       <div>{children}</div>
11     </div>
11   );
13 }
12
   export default Card;
```

children is used here to render whatever we pass inside the component.

### How it works?

1. We define children in the props type (React.ReactNode).
2. The component receives children as a prop.
3. We render {children} inside the JSX.

### Using the Component

```
<Card>
  <h3>Welcome to React!</h3>
  <p>This is passed as children.</p>
</Card>
```

Everything between <Card> and </Card> becomes children.

### Children can be anything

- Text
- JSX elements
- Multiple elements (fragments)
- Other components
- Even a function (render prop style)

Example:

```
<Card>
  <button>Click Me</button>
  <span>Extra content</span>
</Card>
```

All of this will be rendered inside the Card component.

### Quick Note

- React.ReactNode includes:
  - string | number | boolean | null | undefined
  - React elements
  - arrays / fragments
  - portals, etc.

So, children: React.ReactNode is the most flexible type for the children prop.

**In short:** function Card({ children }: { children: React.ReactNode }) => lets us pass any React content into the Card component.

# Props Sending Function Type Safety & Receiving with Function (TypeScript)

TypeScript helps us write safer React code with proper types!

## Page 11 : Passing Functions as Props in React

### What are we doing?

- We are passing a function from a parent component to a child component using props.
- We define the function type in the parent and use the same type in the child to receive it.
- This gives type safety and avoids mistakes like calling a function with wrong arguments.

### Function as a Prop (Overview)

Parent Component (creates function and passes it) → props → Child Component (receives and uses the function)

Both sides use the same function type to keep it safe and consistent.

### Example: Sending a Function as Prop

```tsx
import React from "react";

// Parent Component
const Parent: React.FC = () => {
    const handleClick = (name: string): void => {
        alert(`Hello, ${name}`);
    };

    return (
        <div>
            <h2>Parent Component</h2>
            <Child onClick={handleClick} />
        </div>
    );
};
export default Parent;
```

Function is passed as a prop with a specific type.

### Function Type (in Props)

```tsx
interface ChildProps {
  onClick: (name: string) => void;
}

const Child: React.FC<ChildProps> = ({ onClick }) => {
  return (
    <div>
      <button onClick={() => onClick("React")}>
        Click Me
      </button>
    </div>
  );
};
export default Child;
```

We define the function type in props using a function signature.

### Function Type Safety

- TypeScript checks that the function you pass matches the expected type.
- If you pass a function with wrong parameters or return type, you'll get an error.
- This prevents runtime errors and makes your code more reliable. Safe Code!

### Example: Type Error (Safety)

```tsx
const wrongHandler = (id: number): void => {
    console.log(id);
};

<Child onClick={wrongHandler} />
```

TypeScript catches this before runtime.

Error:
Type '(id: number) => void' is not assignable to type '(name: string) => void'.

### Using the Function Keyword (for receiving)

Instead of writing the full function type, you can use the Function keyword when you don't want to specify parameters or return type (less safe, but sometimes used).

```tsx
interface ChildProps {
  onClick: Function;
}
```

This means:
- it's a function
- but we don't care about parameters or return type.

⚠ Note: It is less strict, so prefer specific function types like (name: string) => void for better safety.

### Receiving with Function Type (Alternative)

```tsx
type ClickHandler = (name: string) => void;
interface ChildProps {
  onClick: ClickHandler;    // using a type alias
}
```

This is another clean way to keep your function type reusable and readable.

### Quick Note

- Use specific function types for better safety.
- Use the Function keyword only if you don't need parameter/return type details.
- Always match the function type on both sides.

**In short:** Use function types in props to pass functions safely, let TypeScript check the types, and avoid runtime errors.

Type Safety = Better Code